



Universidad Nacional de Colombia

FACULTAD DE CIENCIAS

MATERIAL SUPLEMENTARIO - PROYECTO

Matematicas del Aprendizaje de Maquinas
Prof. Francisco Albeiro Gomez Jaramillo

Presentado por:

Luis Andrés Rodríguez Florián , Angel David Gomez Pastrana.

Bogotá, Julio del 2026

1. Introducción

La deforestación representa una de las principales amenazas para el cambio climático y la pérdida de biodiversidad a escala global, afectando severamente la regulación de los ciclos hidrológicos, las emisiones de gases de efecto invernadero y la capacidad de captura de carbono. La conversión de bosques hacia usos agropecuarios, la expansión de la infraestructura y las actividades extractivas reducen y fragmentan ecosistemas de alto valor ecológico. En Colombia, esta problemática adquiere una relevancia particular debido a la importancia ecológica de la Amazonía y de otros bosques tropicales. El Sistema de Monitoreo de Bosques y Carbono (SMBYC) del IDEAM publica periódicamente boletines de Detección Temprana de Deforestación (DTD), los cuales evidencian que la deforestación continúa concentrándose principalmente en la región Amazónica y resaltan la urgencia de desarrollar herramientas capaces de anticipar nuevos focos de pérdida de bosque [1].

Durante las últimas décadas, la disponibilidad de grandes volúmenes de datos satelitales y el desarrollo de técnicas de percepción remota han transformado significativamente el monitoreo forestal. Plataformas como Landsat y Sentinel han permitido observar de manera continua y automatizada la evolución de la cobertura vegetal incluso en regiones remotas [2]. No obstante, estos sistemas presentan una limitación inherente: las alertas son emitidas una vez que el evento de deforestación ya ha ocurrido, por lo que constituyen herramientas esencialmente reactivas.

Es por esto que herramientas de detección temprana, que permitan modelar la evolución y hacer pronósticos de áreas en riesgo de deforestación son útiles, puesto que permitan a agentes gubernamentales y autoridades regionales tomar decisiones adecuadas y oportunas, como también llevar a cabo la distribución efectiva de recursos para la protección de los bosques y ecosistemas en riesgo. Sin embargo la tarea de realizar pronósticos sobre la evolución de eventos como la deforestación es compleja, pues es un fenómeno que depende de múltiples factores, como la actividad humana, la geografía, el clima y dinámicas socioeconómicas de la región (tala, minería, agricultura, etc).

Por este motivo en este proyecto decidimos abordar este problema y reproducir el artículo de [3] (Using deep convolutional neural networks to forecast spatial patterns of Amazonian deforestation) con el propósito de adaptar estos modelos a regiones de Colombia, así como analizar la metodología, las arquitecturas de los modelos utilizados y las métricas de evaluación para resolver el problema.

El artículo emplea modelos de aprendizaje profundo específicamente Redes Neuronales Convolucionales Profundas (CNNs), estas han mostrado ser efectivas en aprender las dinámicas y características espaciales presentes en los entornos naturales capturados por las imágenes satelitales. Estas fueron entrenadas utilizando datos del *Global Forest Change Dataset* y el *Japan Aerospace Exploration Agency's (JAXA)*, para resolver el problema, este es planteado como una extensión temporal de el problema de clasificación de cobertura de suelos.

Así, nuestros objetivos y el curso de trabajo serán los siguientes. En primer lugar, exploraremos y entenderemos los conjuntos de datos utilizados con el fin de plantear de manera formal el problema de aprendizaje, describiendo las variables de entrada y de salida. Posteriormente, estudiaremos las diferentes arquitecturas empleadas, incluyendo redes neuronales convolucionales bidimensionales (2DCNN), tridimensionales (3DCNN) y redes convolucionales recurrentes (ConvLSTM), con el propósito de comprender su funcionamiento y sus diferencias. Asimismo, explicaremos las métricas utilizadas para evaluar el desempeño de estos modelos. Luego, analizaremos los resultados reportados en [3]. Finalmente, buscaremos extender los modelos desarrollados al contexto colombiano, evaluando su aplicabilidad y adaptación a las características

de los datos nacionales; para ello, nos guiaremos por los boletines publicados por el SMByC del IDEAM [1] y seleccionaremos como caso de estudio el núcleo de deforestación 8, correspondiente al municipio de Mapiripán, Meta que. De acuerdo con el Boletín de Detección Temprana de Deforestación (DTD), este núcleo se localiza en el sector nororiental de Mapiripán, abarcando los bosques delimitados por los ríos Iteviare y Siare. Cuya deforestación es impulsada por causas como la praderización para acaparamiento de tierras, prácticas no sostenibles de ganadería extensiva, vías de acceso no planeadas y expansión agrícola de pequeña escala.

2. Datos

En esta sección se presentan los principales conjuntos de datos geoespaciales utilizados en el trabajo *Using Deep Convolutional Neural Networks to Forecast Spatial Patterns of Amazonian Deforestation* [3]. En primer lugar, se describe el *Global Forest Change Dataset*, empleado para identificar los eventos históricos de pérdida de cobertura forestal utilizados durante el entrenamiento del modelo. Posteriormente, se presenta el *JAXA ALOS Global Digital Surface Model (DSM)*, del cual se obtiene la información topográfica utilizada como variable explicativa. Finalmente, se analiza la forma en que ambos conjuntos de datos son integrados dentro de la metodología propuesta por los autores.

2.1. Global Forest Change Dataset

El *Global Forest Change (GFC) Dataset*, desarrollado por Hansen et al. [2], proporciona mapas globales de la cobertura arbórea y de sus cambios desde el año 2000 con una resolución espacial aproximada de 30 metros por pixel. El conjunto de datos fue construido a partir de imágenes adquiridas por la misión Landsat y constituye uno de los productos de referencia para el monitoreo de la dinámica forestal y el análisis de cambios en la cobertura terrestre.

A diferencia de una colección de imágenes satelitales sin procesar, el GFC corresponde a un producto derivado obtenido mediante una cadena de procesamiento que incluye el preprocesamiento de imágenes Landsat, la extracción de variables espectrales, la generación de composiciones multitemporales libres de nubes y la clasificación de la cobertura arbórea mediante modelos de *Bagged Decision Trees*. A partir de estas composiciones temporales, el algoritmo compara las observaciones disponibles para identificar cambios en la cobertura arbórea y determinar el año en que ocurrió cada evento de pérdida [2].

El resultado de este procesamiento se distribuye mediante diferentes capas raster, cada una de las cuales representa una característica específica de la superficie terrestre. Una capa raster corresponde a una matriz de píxeles georreferenciados donde cada celda almacena el valor de una variable para una ubicación determinada. En la versión 1.8 del Global Forest Change, utilizada durante el periodo de publicación del trabajo analizado, las principales capas son:

- **treecover2000:** almacena el porcentaje de cobertura arbórea estimada para el año 2000. La cobertura arbórea corresponde a vegetación con una altura superior a cinco metros. Por ejemplo, un valor de 87 indica que aproximadamente el 87 % del área representada por ese píxel estaba cubierta por árboles en el año 2000 [4].
- **gain:** identifica los píxeles donde ocurrió una transición de no bosque a bosque durante el periodo 2000–2012. Es una variable binaria donde 1 representa ganancia de cobertura

forestal y 0 indica ausencia de ganancia. Es importante destacar que esta capa no se actualiza después de 2012 [4].

- **lossyear**: registra el año en que se detectó una pérdida de cobertura arbórea. Un valor de 0 indica que no se detectó pérdida, mientras que los valores entre 1 y 20 representan pérdidas ocurridas entre los años 2001 y 2020, respectivamente. Por ejemplo, un valor de 15 corresponde a una pérdida detectada durante el año 2015 [4].
- **datamask**: clasifica cada píxel según el tipo de superficie. Los valores posibles son 0 (sin datos), 1 (superficie terrestre) y 2 (cuerpos de agua permanentes) [4].
- **first**: composición multiespectral libre de nubes correspondiente aproximadamente al inicio del periodo de estudio (año 2000), utilizada como referencia para representar el estado inicial de la cobertura terrestre [4].
- **last**: composición multiespectral libre de nubes correspondiente al último año disponible del producto (2020 en la versión utilizada), empleada para representar el estado final de la cobertura terrestre y facilitar la detección de cambios [4].

Al compartir la misma resolución espacial y sistema de referencia, estas capas pueden superponerse y analizarse conjuntamente, permitiendo relacionar diferentes características del territorio para una misma ubicación geográfica. La documentación oficial del proyecto describe detalladamente cada una de las capas disponibles, mientras que una visualización interactiva del conjunto de datos puede consultarse mediante el visor *Global Forest Change* desarrollado por GLAD [4, 5].

Es importante destacar que el GFC identifica eventos de *tree cover loss* (pérdida de cobertura arbórea) y no necesariamente procesos permanentes de deforestación. La pérdida de cobertura arbórea puede estar asociada a diferentes causas, como el aprovechamiento forestal, incendios, fenómenos naturales o cambios en el uso del suelo. Por esta razón, el conjunto de datos constituye una medida objetiva de los cambios observados sobre la cobertura arbórea, mientras que la interpretación de dichos cambios depende del contexto específico del estudio [2].

El Global Forest Change es un producto que continúa actualizándose periódicamente. La versión empleada durante el desarrollo del trabajo analizado incorpora información hasta el año 2020; sin embargo, versiones más recientes han extendido la serie temporal hasta el año 2025, manteniendo la misma estructura del conjunto de datos y actualizando principalmente la capa *lossyear*, que registra nuevos eventos de pérdida de cobertura arbórea, así como la composición multitemporal *last*, utilizada como representación del estado final de la superficie terrestre [6].

2.2. JAXA ALOS Global Digital Surface Model

El segundo conjunto de datos utilizado en el trabajo corresponde al *JAXA ALOS Global Digital Surface Model (DSM)*, conocido oficialmente como *ALOS World 3D - 30 m (AW3D30)*. Este producto, desarrollado por la *Japan Aerospace Exploration Agency (JAXA)*, proporciona un modelo digital de superficie con cobertura global y una resolución espacial aproximada de 30 metros, generado a partir de imágenes estereoscópicas adquiridas por el sensor PRISM a bordo del satélite ALOS [7, 8].

A diferencia del *Global Forest Change Dataset*, cuyo propósito es describir los cambios en la cobertura arbórea, el AW3D30 representa las características topográficas de la superficie terrestre. El producto incluye diferentes bandas y archivos auxiliares que contienen información sobre la elevación de la superficie, la calidad de los datos y el proceso de generación del modelo. Entre las principales capas distribuidas se encuentran:

- **DSM**: almacena la elevación de la superficie respecto al nivel medio del mar, expresada en metros. Cada píxel contiene la altura correspondiente a esa ubicación geográfica. Por ejemplo, un valor de 350 indica que la superficie representada por el píxel se encuentra aproximadamente a 350 metros sobre el nivel del mar [9].
- **MSK**: corresponde a una máscara de calidad que identifica diferentes condiciones presentes durante la generación del modelo, como cuerpos de agua, nubes o regiones completadas mediante otras fuentes de información, permitiendo evaluar la confiabilidad de cada píxel [9].
- **STK**: indica el número de observaciones estereoscópicas utilizadas para estimar la elevación de cada píxel. En general, un mayor número de observaciones proporciona una estimación más robusta de la altura de la superficie [9].

Aunque el conjunto de datos contiene información adicional sobre la calidad del producto y el proceso de generación del modelo, el trabajo *Using Deep Convolutional Neural Networks to Forecast Spatial Patterns of Amazonian Deforestation* emplea únicamente la información topográfica derivada de la banda **DSM**. En particular, los autores utilizan la variable de *elevación* obtenida directamente del modelo digital de superficie y calculan a partir de ella la variable de *pendiente* (*slope*), la cual describe la inclinación del terreno y constituye un atributo relevante para caracterizar la accesibilidad y la probabilidad de ocurrencia de procesos de deforestación [3].

La incorporación de estas variables permite complementar la información proveniente del *Global Forest Change Dataset*, ya que mientras este último describe la ocurrencia de eventos históricos de pérdida de cobertura arbórea, el AW3D30 aporta información sobre las características físicas del terreno. De esta manera, el modelo de aprendizaje profundo puede considerar simultáneamente la distribución espacial de la deforestación y la influencia que ejerce la topografía sobre dichos procesos.

2.3. Uso de los conjuntos de datos en el modelo

Los conjuntos de datos descritos en las secciones anteriores no son utilizados directamente como entrada de las redes neuronales. En su lugar, los autores realizan un proceso de extracción de características (*feature extraction*) mediante el cual la información contenida en el *Global Forest Change Dataset* y en el *JAXA ALOS Global Digital Surface Model* se transforma en un conjunto de variables predictoras que posteriormente son organizadas en tensores para el entrenamiento de los diferentes modelos de aprendizaje profundo [3].

Como punto de partida, los autores recopilaron diez archivos raster provenientes del *Global Forest Change Dataset*: las capas *treecover2000*, *gain*, *datamask*, la capa *lossyear* correspondiente a la versión 2020 del producto y las siete composiciones multiespectrales anuales *last* comprendidas entre los años 2014 y 2020. Adicionalmente, incorporaron la banda *DSM* del producto *JAXA ALOS Global Digital Surface Model*, obteniendo así la información necesaria para construir las variables predictoras utilizadas por el modelo [3].

Las variables predictoras construidas a partir de ambos conjuntos de datos se resumen en la Tabla 1, estas pueden clasificarse en dos grupos: variables estáticas y variables dinámicas. Las variables estáticas permanecen constantes durante todo el periodo de análisis e incluyen *treecover2000* y *datamask*, obtenidas directamente del *Global Forest Change Dataset*. De manera opcional, los autores también evaluaron la incorporación de la variable *elevation*, obtenida directamente de la banda *DSM* del producto AW3D30. Sin embargo, los experimentos mostraron

Variable	Dataset	Origen
treecover2000	GFC	treecover2000
datamask	GFC	datamask
elevation ^a	AW3D30	DSM
recent_loss1	GFC	lossyear
recent_loss2	GFC	lossyear
recent_loss3	GFC	lossyear
recent_loss4	GFC	lossyear
last_b30	GFC	last (Band 3)
last_b40	GFC	last (Band 4)
last_b50	GFC	last (Band 5)
last_b70	GFC	last (Band 7)

^a Variable evaluada experimentalmente; no fue incluida en la configuración final del modelo.

Cuadro 1: Relación entre las variables predictoras utilizadas por el modelo y los conjuntos de datos de origen. Adaptado de [3].

que esta variable no producía mejoras significativas en el desempeño de los modelos, por lo que no fue incluida en la configuración final [3].

Las variables dinámicas describen la evolución temporal de cada píxel. Para ello, la información contenida en la capa *lossyear* no se utiliza directamente; en su lugar, se generan cuatro variables binarias denominadas *recent_loss1*, *recent_loss2*, *recent_loss3* y *recent_loss4*. Cada una resume la ocurrencia de pérdida de cobertura arbórea en un intervalo temporal distinto: *recent_loss1* representa pérdidas ocurridas entre los años $[t, t-2)$, *recent_loss2* entre $[t-2, t-5)$, *recent_loss3* entre $[t-5, t-8)$ y *recent_loss4* agrupa todas las pérdidas ocurridas con anterioridad a ese intervalo hasta el año 2000. Esta representación permite conservar el historial reciente de deforestación sin emplear directamente el año exacto almacenado en la variable *lossyear* [3].

Además del historial de deforestación, el modelo incorpora información espectral proveniente de las composiciones multispectrales anuales *last*. Para cada año t , los autores extraen las bandas Landsat 7 correspondientes al rojo (*Band 3*), infrarrojo cercano (*Band 4*), infrarrojo de onda corta 1 (*Band 5*) e infrarrojo de onda corta 2 (*Band 7*), las cuales conforman las variables *last_b30(t)*, *last_b40(t)*, *last_b50(t)* y *last_b70(t)* descritas en la Tabla 1 del material suplementario. Estas bandas corresponden a la observación libre de nubes más reciente disponible para cada píxel durante el año considerado [3, 4].

Una vez construidas las variables predictoras, la información se reorganiza en ventanas espaciales centradas sobre cada píxel objetivo. En lugar de utilizar únicamente la información correspondiente al píxel que se desea clasificar, los autores extraen una ventana cuadrada de tamaño $(2r+1) \times (2r+1)$, donde el píxel central corresponde al punto cuya deforestación se pretende predecir y los píxeles vecinos proporcionan el contexto espacial necesario para que la red neuronal aprenda patrones espaciales asociados a la dinámica de la deforestación [3].

Cada muestra de entrenamiento queda representada mediante dos tensores. El primero, denominado *Static Tensor*, contiene las variables estáticas y posee dimensiones $C_s \times (2r+1) \times (2r+1)$, donde $C_s = 2$ cuando únicamente se consideran las capas *treecover2000* y *datamask*, o $C_s = 3$ cuando también se incorpora la variable de elevación. El segundo corresponde a una secuencia temporal de tensores dinámicos (*Dynamic Tensor*), en la que cada instante temporal está formado por ocho canales: las cuatro variables *recent_loss* y las cuatro bandas espectrales Landsat (*last_b30*, *last_b40*, *last_b50* y *last_b70*). Esta representación conserva simultáneamente la información espacial, espectral y temporal necesaria para entrenar las arquitecturas 2D CNN, 3D

CNN y ConvLSTM evaluadas en el trabajo [3].

Finalmente, a cada muestra se le asigna una etiqueta binaria Y_{t+1} , cuyo objetivo consiste en indicar si el píxel central experimentará un evento de pérdida de cobertura arbórea durante el año inmediatamente posterior al último instante observado. En particular, la etiqueta toma el valor 1 únicamente cuando el píxel es registrado como pérdida de cobertura arbórea exactamente en el año $t + 1$, mientras que toma el valor 0 si la pérdida ocurrió en cualquier otro año del periodo de estudio o si el píxel nunca presentó un evento de deforestación. De esta forma, el problema se formula como una tarea de clasificación binaria de predicción a un año (*one-year ahead forecasting*) [3].

Antes de construir el conjunto de entrenamiento, los autores restringen el análisis únicamente a aquellos píxeles que pueden representar bosque durante el año t . Esta decisión se fundamenta en las propiedades del Global Forest Change Dataset. En dicho conjunto de datos, una vez que un píxel es registrado como pérdida de cobertura arbórea, éste no vuelve a clasificarse como bosque en años posteriores. Además, únicamente se consideran como bosque aquellos píxeles cuya cobertura arbórea inicial es superior al 30 %, salvo que hayan presentado un evento de ganancia (*gain*) durante el periodo 2001–2012. Finalmente, los píxeles correspondientes a cuerpos permanentes de agua, identificados mediante la capa *datamask*, son excluidos del análisis.

En consecuencia, el conjunto de píxeles candidatos para realizar la predicción en el año $t + 1$ se define como aquellos que aún permanecen forestados en el año t , descartando regiones previamente deforestadas, cuerpos de agua y áreas que nunca correspondieron a bosque. Esta restricción reduce el espacio de búsqueda y evita realizar predicciones sobre ubicaciones donde un nuevo evento de deforestación resulta imposible por construcción del conjunto de datos [3].

Antes del entrenamiento, las variables continuas fueron normalizadas para llevarlas a un rango común. En particular, la variable *treecover2000*, cuyo rango original corresponde a valores entre 0 y 100, y las bandas espectrales Landsat, cuyos valores se encuentran entre 0 y 255, fueron reescaladas al intervalo $[0, 1]$. Este proceso facilita el entrenamiento de las redes neuronales al evitar diferencias significativas de escala entre las distintas variables de entrada [3].

Los autores observaron que menos del 0.5 % de los píxeles experimentaban un evento de deforestación en un año determinado, generando un fuerte desbalance entre las clases. Para evitar que el modelo aprendiera a predecir únicamente la clase mayoritaria, se aplicó una estrategia de submuestreo (*undersampling*) sobre la clase negativa, manteniendo aproximadamente una relación de un píxel deforestado por cada cuatro píxeles no deforestados durante el entrenamiento. A pesar de esta reducción, cada región de estudio continuó disponiendo de varios millones de muestras para entrenar los modelos [3].

3. Métricas

En esta sección se describen las métricas que se usaron para evaluar los modelos de predicción de deforestación en el trabajo de Ball et al. [3]. Se presenta el propósito de cada métrica, la información que aporta sobre el desempeño del modelo y las razones por las cuales resulta adecuada para este problema. Asimismo, se explica porque otras métricas conocidas no se tuvieron en cuenta.

3.1. AUC (Área bajo la curva ROC)

La curva ROC (*Receiver Operating Characteristic*) representa la relación entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR) para distintos umbrales de decisión. El área bajo esta curva, denominada ROC-AUC (*Area Under the Curve*), se define como

$$AUC = \int_0^1 TPR(FPR) d(FPR).$$

Desde una perspectiva probabilística, el ROC-AUC también puede interpretarse como

$$AUC = P(s(x^+) > s(x^-)),$$

donde $s(x)$ representa la puntuación asignada por el modelo, x^+ una muestra positiva y x^- una muestra negativa. Esta expresión indica la probabilidad de que el clasificador otorgue una puntuación mayor a una muestra positiva que a una negativa seleccionadas aleatoriamente.

3.2. Precision

La precisión (*Precision*) mide la proporción de predicciones positivas que realmente corresponden a eventos positivos. Matemáticamente se expresa como

$$Precision = \frac{TP}{TP + FP}.$$

Un valor elevado de precisión indica que la mayoría de las áreas clasificadas como deforestadas corresponden efectivamente a eventos reales, reduciendo el número de falsas alarmas. En aplicaciones de monitoreo ambiental, una alta precisión disminuye la cantidad de recursos destinados a inspeccionar zonas incorrectamente identificadas como deforestadas.

3.3. Recall

La sensibilidad o *Recall* mide la capacidad del modelo para detectar los eventos positivos existentes. Se define como

$$Recall = \frac{TP}{TP + FN}.$$

En el contexto de la predicción de deforestación, esta métrica representa la proporción de áreas realmente deforestadas que fueron correctamente identificadas por el modelo. Un alto valor de *Recall* es especialmente importante debido a que los falsos negativos implican eventos de deforestación que pasan inadvertidos y, por tanto, no podrían ser atendidos oportunamente.

3.4. F1-Score

El *F1-Score* combina las métricas de precisión y sensibilidad mediante su media armónica,

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

Esta métrica proporciona un equilibrio entre ambas medidas, penalizando aquellos modelos que presentan un desempeño muy alto en una de ellas pero bajo en la otra. Debido al desbalance de clases presente en los problemas de detección de deforestación, el *F1-Score* constituye un indicador más representativo que la exactitud para comparar distintos modelos de clasificación.

3.5. ¿Y las otras métricas?

La selección de las métricas de evaluación depende de las características del problema de clasificación. En la detección de deforestación, la cantidad de píxeles correspondientes a áreas no deforestadas es considerablemente mayor que la de píxeles deforestados, lo que da lugar a un conjunto de datos con clases desbalanceadas. En este contexto, métricas como la *Accuracy* pueden proporcionar una estimación optimista del desempeño del modelo, ya que un clasificador podría obtener valores elevados prediciendo mayoritariamente la clase negativa. De manera similar, métricas como la especificidad (*Specificity*) o el valor predictivo negativo (*Negative Predictive Value*) se centran principalmente en evaluar el desempeño sobre la clase negativa, la cual representa la mayoría de las observaciones. Por esta razón, Ball et al. [3] emplean las métricas *Precision*, *Recall*, *F1-Score* y ROC-AUC, ya que estas permiten evaluar de forma más adecuada la capacidad del modelo para identificar correctamente los eventos de deforestación, controlar la cantidad de falsas alarmas y medir su capacidad discriminativa entre ambas clases.

4. Modelos

En esta sección presentamos las diferentes arquitecturas de redes neuronales convolucionales utilizadas por los autores para abordar el problema de predicción de deforestación. Estas arquitecturas incluyen la red neuronal convolucional 2D (2D CNN), la red neuronal convolucional 3D (3D CNN) y la red convolucional recurrente (ConvLSTM).

La predicción de la deforestación constituye un problema de clasificación espacio-temporal cuyo objetivo consiste en estimar la probabilidad de que una determinada región del bosque experimente pérdida de cobertura forestal durante un horizonte temporal futuro. A diferencia de los problemas clásicos de clasificación supervisada, en los cuales las observaciones suelen considerarse independientes, la deforestación presenta una fuerte dependencia tanto espacial como temporal.

Todas estas arquitecturas antes mencionadas fueron diseñadas para explotar simultáneamente la información espacial y temporal contenida en secuencias de imágenes Landsat, permitiendo modelar la evolución de la cobertura forestal y predecir la ocurrencia de eventos futuros de deforestación. Antes de describir cada una de estas arquitecturas resulta conveniente revisar los componentes fundamentales que las conforman. En particular, todas ellas están construidas sobre Redes Neuronales Convolucionales (CNN), las cuales constituyen el mecanismo principal para la extracción automática de características espaciales.

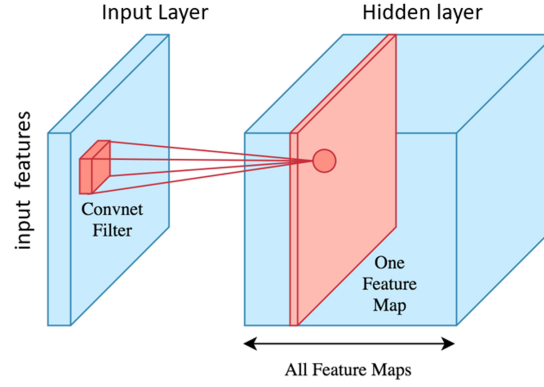


Figura 1: Un filtro en una red neuronal Convolutiva 2D tiene forma 3D. Se desliza a través de la altura y ancho del tensor 3D de entrada (características) y por lo tanto es conectado únicamente con una región local de la entrada en cada instante. Realiza una operación de producto interno de sus pesos con las entradas del tensor en esa posición y el resultado es pasado por una función de activación y la activación de este valor es enviado a un mapa de activaciones 2D (mapa de características). Tomado de [10]

4.1. Redes Neuronales Convolucionales

Las Redes Neuronales Convolucionales (*Convolutional Neural Networks*, CNN) fueron desarrolladas específicamente para el procesamiento de datos organizados sobre una malla regular, como las imágenes digitales. Su principal ventaja consiste en aprender automáticamente representaciones jerárquicas mediante operaciones de convolución, permitiendo extraer características espaciales relevantes.

A diferencia de las redes neuronales completamente conectadas, donde cada neurona recibe información proveniente de todas las variables de entrada, las CNN restringen las conexiones entre neuronas a regiones locales de la imagen mediante filtros convolucionales de tamaño reducido, explotando así las relaciones espaciales de los datos. Estos filtros son deslizados (convolucionados) sobre las dimensiones de la entrada realizando la operación únicamente en las regiones locales correspondientes, y el resultado de cada región es guardado en un mapa de activación (*activation map*), cada filtro genera un mapa de activación distinto.

4.2. Redes Neuronales Convolucionales 2D

La entrada de una 2DCNN es una imagen con múltiples canales, estos canales son superpuestos generando un tensor $X \in \mathbb{R}^{H \times W \times C}$, donde H es la altura de la imagen, W es el ancho de la imagen y C son los canales (ej. Los canales RGB de una imagen a color). El filtro que realiza la operación de la convolución está compuesto por pesos y un sesgo que son entrenables, el conjunto de los pesos es llamado núcleo y es de la forma $\Omega \in \mathbb{R}^{k \times k \times C}$, donde k es el tamaño del núcleo siendo un número impar para poder centrar la operación de convolución sobre una entrada particular de la imagen $X(i, j)$ y sobre todos los canales. Así el filtro es deslizado sobre el ancho y el alto de la imagen aplicando una operación de producto interno, y cuya salida es alimentada activada y guardada en el mapa de activación, esta operación se ve de la siguiente manera:

$$a\left(\sum_{m=1}^k \sum_{n=1}^k \sum_{c=1}^C \Omega(m, n, c) X(i+u, j+v, c) + \beta, \right)$$

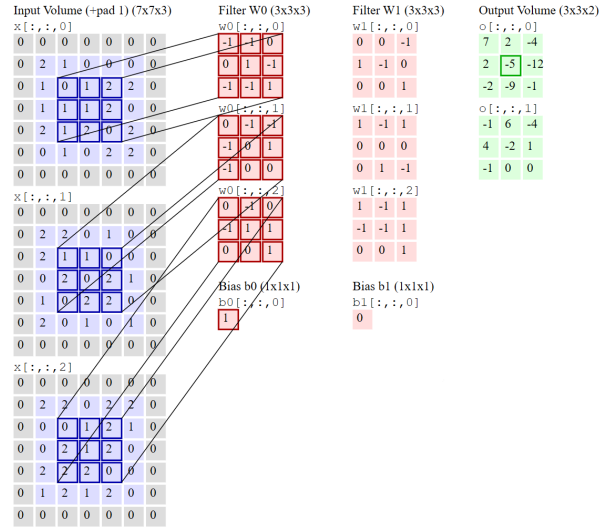


Figura 2: Convolución de una imagen con 3 canales utilizando 2 filtros. Tomado de [11]

Aquí $\Omega(m, n, c)$ representa el peso (m, n, c) del núcleo, β el sesgo asociado al filtro, cada núcleo tiene un sesgo asociado.

Cada filtro produce una única imagen de salida denominada *mapa de activación* (*feature map*). Si una capa convolucional contiene F filtros distintos, $K_1, K_2, \dots, K_F$, entonces la salida completa de la capa está formada por F mapas de activación. En consecuencia, el tensor de salida posee dimensiones $H_{\text{out}} \times W_{\text{out}} \times F$. Es importante notar que la profundidad del tensor de salida ya no depende del número de canales originales de la imagen, sino del número de filtros aprendidos por la red. Cada mapa de activación representa la presencia espacial de una característica específica aprendida durante el entrenamiento. El parámetro denominado *stride* controla el desplazamiento del filtro entre dos posiciones consecutivas. Si el stride es igual a uno, $s = 1$, el filtro analiza todas las posiciones posibles de la imagen. Por el contrario, cuando $s > 1$, el filtro avanza varios píxeles en cada desplazamiento, reduciendo la resolución espacial del mapa de activación. La dimensión espacial de la salida viene dada por

$$H_{\text{out}} = \left\lfloor \frac{H - k_h + 2P}{s} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W - k_w + 2P}{s} \right\rfloor + 1,$$

donde P representa el tamaño del padding, s corresponde al stride, k_h y k_w son las dimensiones espaciales del filtro.[11]

La figura 2 ilustra una convolución 2D aplicada a una imagen. La imagen tiene tamaño $5 \times 5 \times 3$ (3 canales), se aplicó un relleno de tamaño 1 que hace la imagen $7 \times 7 \times 3$. En la figura 2 están presentes núcleos de tamaño 3×3 . Cada uno tiene sus respectivos pesos y sesgo asociado. Adicionalmente la convolución se realizó con un salto (*o stride*) 2. Esto produce una salida de tamaño $3 \times 3 \times 2$.

4.3. Redes Neuronales Convolucionales 3D

Las Redes Convolucionales 3D se extienden incorporando una dimensión adicional en la operación de convolución. La entrada es ahora un tensor 4D cuyas dimensiones corresponden a el

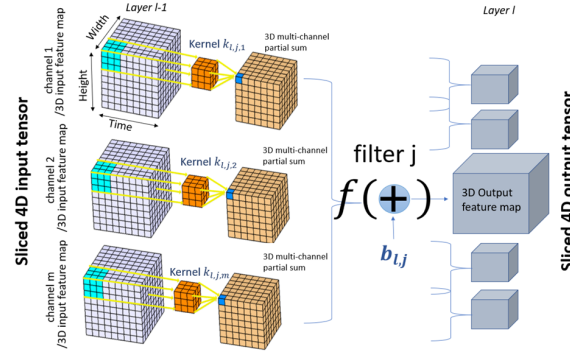


Figura 3: Convolucion 3D de un tensor 4D. Tomado de [10]

tiempo, la altura, el ancho y los canales. Mientras que una CNN 2D utiliza filtros bidimensionales, una 3D CNN emplea filtros de dimensiones correspondientes, $t \times k \times k \times C$, (donde tenemos ahora una adicional temporal especifica para nuestro caso). Las convoluciones 3D permiten que los filtros se desplacen simultáneamente sobre las dimensiones espaciales y temporal, aprendiendo características espacio-temporales directamente durante el proceso de entrenamiento. La operación de convolución tridimensional puede escribirse como

$$a\left(\sum_{\tau=1}^t \sum_{u=1}^{2r-1} \sum_{v=1}^{2r-1} \sum_{c=1}^C \Omega(\tau, u, v, c) X(t + \tau, i + u, j + v, c) + \beta\right)$$

4.4. Redes Neuronales Convolucionales Profundas

De manera similar que una Red neuronal tradicional las CNN son profundas cuando tienen mas de una capa convolucional, anteriormente describimos el funcionamiento de una capa actuando directamente sobre la entrada. Para las demás capas convolucionales ocultas, la entrada sera el conjunto apilado de mapas de activación correspondientes a cada filtro. Asi para una Red Neuronal Convolucional Profunda 2D se tiene la siguiente ecuación:

$$v_{j,x,y}^l = a\left(\sum_{u=1}^k \sum_{v=1}^k \sum_{m=1}^M \omega_{j,u,v,m}^l v_{x+u,y+v,m}^{l-1} + \beta_j^l\right)$$

Donde l denota la capa donde llega la nueva activación v , j corresponde al filtro o el numero de mapas de activación de la capa l , M es el numero de capas de la entrada o el numero de mapas de activación de la capa anterior $l - 1$. x, y corresponde a las coordenadas espaciales. $k_{j,u,v,m}^l$ corresponde el peso u, v, m del j -esimo filtro en la capa l y b_j^l el sesgo correspondiente.

4.5. Batch Normalization

A medida que aumenta la profundidad de una red neuronal, el proceso de entrenamiento se vuelve considerablemente más difícil. Una de las principales causas de este fenómeno es que la distribución de las activaciones internas cambia continuamente durante el entrenamiento conforme se actualizan los parámetros de las capas anteriores. Como consecuencia, cada capa debe adaptarse constantemente a nuevas distribuciones de entrada, lo que ralentiza la convergencia y dificulta la optimización mediante descenso por gradiente.

Para atacar este problema Ioffe y Szegedy (2015) [12] introducen la tecnica de *Batch Normalization* (Normalización por Lotes) que consiste en una capa que normaliza cada filtro de la red para que tenga una media de cero y una varianza unitaria. Ellos demostraron que el uso de estas capas en redes neuronales aporta múltiples ventajas: Por un lado permite que el algoritmo de descenso de gradiente utilice tasas de aprendizaje (learning rates) más altas. Además la convergencia de la función de pérdida se vuelve significativamente menos sensible a la inicialización de los pesos. Y por último ofrece cierto nivel de regularización al añadir un pequeño ruido a los datos. En ocasiones, este efecto es tan beneficioso que puede igualar el rendimiento del dropout, disminuyendo la necesidad de incluir capas de dropout en la red.

En una CNN, una capa de Batch Normalization 2D normaliza las entradas de cada mapa de características antes de su activación. Esta normalización se realiza calculando la media y la varianza estimadas en todas las ubicaciones y lotes (*batches*). Considerando un *minibatch* que tiene m tensores 3D (altura, anchura y profundidad) convolucionados con d filtros, la salida de esta capa es un bloque de d mapas de características 2D. Durante el entrenamiento, cada elemento (l, k) del j -ésimo mapa de características, procesado a partir de la imagen b en el lote, se transforma de la siguiente manera:

$$\hat{y}_{b,j,l,k} = \hat{x}_{b,j,l,k} \times \gamma + \beta$$

$$\hat{x}_{b,j,l,k} = \frac{x_{b,j,l,k} - \hat{E}[x_j]_{Moving}}{\hat{\sigma}_{Moving_j} + \epsilon}$$

Donde γ y β son parámetros aprendibles de escala y desplazamiento espacial, y ϵ es una constante añadida para garantizar la estabilidad numérica. Durante la inferencia, las medias móviles empíricas ($\hat{E}[x_j]_{Moving}$) y la varianza ($\hat{\sigma}_{Moving_j}$) a lo largo de los lotes se mantienen como constantes. Para calcular la media empírica ($\hat{E}[x_j]$) y la varianza ($\hat{\sigma}^2[x_j]$) del lote, se emplean las siguientes sumatorias sobre el tamaño espacial $H_j \times W_j$ del mapa de características:

$$\hat{E}[x_j] = \sum_{i=0}^m \sum_{h=0}^{H_j} \sum_{w=0}^{W_j} x_{i,j,h,w}$$

$$\hat{\sigma}^2[x_j] = \frac{1}{m \times H_j \times W_j} \sum_{i=0}^m \sum_{h=0}^{H_j} \sum_{w=0}^{W_j} (x_{i,j,h,w} - \hat{E}[x_j])^2$$

Las ecuaciones para la Normalización por Lotes en 3D son análogas al caso 2D, con la diferencia principal de que los mapas de características son tensores 3D, lo que introduce la dimensión temporal T_j :

$$\hat{E}[x_j] = \sum_{i=0}^m \sum_{t=0}^{T_j} \sum_{h=0}^{H_j} \sum_{w=0}^{W_j} x_{i,j,t,h,w}$$

$$\hat{\sigma}^2[x_j] = \frac{1}{m \times T_j \times H_j \times W_j} \sum_{i=0}^m \sum_{t=0}^{T_j} \sum_{h=0}^{H_j} \sum_{w=0}^{W_j} (x_{i,j,t,h,w} - \hat{E}[x_j])^2$$

En su artículo original, Ioffe y Szegedy (2015) aplicaron la Normalización por Lotes calculando primero la normalización de las entradas y luego aplicando la función de activación no lineal $f(\cdot)$ (como ReLU). No obstante, el orden correcto de aplicación entre la activación y la normalización sigue siendo un tema de debate en la literatura. Para este estudio, tras diversas pruebas empíricas, se determinó que las redes logran un mejor rendimiento cuando la activación se aplica antes de la capa de normalización, es decir: $y = BN[ReLU(x)]$.

4.5.1. Dropout

La técnica de Dropout fue propuesta por Srivastava et al. (2014) [13] como un método de regularización para redes neuronales. A medida que las redes se vuelven más profundas, el número de pesos crece exponencialmente, lo que puede causar un sobreajuste (*overfitting*) si no se emplean medidas de regularización adecuadas. Durante el entrenamiento, el *Dropout* multiplica las activaciones ocultas por una variable aleatoria de Bernoulli, la cual toma el valor de 0 con una probabilidad p , o de 1 con una probabilidad $1 - p$. Al apagar aleatoriamente diferentes unidades neuronales en cada *forward pass*, la red pasa a representar múltiples subredes. Esto previene la coadaptación, ya que la red no puede depender de ninguna neurona en particular, evitando que la salida final dependa de una única característica aprendida. Durante la inferencia, se requiere que el modelo utilice todos los pesos aprendidos sin descartar ninguno. Para mantener la escala, las puntuaciones de cada neurona se multiplican por la probabilidad de no ser descartada, es decir, $1 - p$. Considerando un vector de características de activaciones $x = (x_1, x_2, \dots, x_d)$. Al aplicar *Dropout* durante el entrenamiento, el vector se transforma en:

$$x = (a_1x_1, a_2x_2, \dots, a_dx_d)$$

Donde a_k son variables aleatorias independientes de Bernoulli. Durante la fase de prueba o inferencia, el vector se ajusta de la siguiente manera:

$$x = ((1 - p)x_1, (1 - p)x_2, \dots, (1 - p)x_d)$$

Alternativamente (como se implementa en **PyTorch**), las activaciones pueden escalarse multiplicándolas por $\frac{1}{1-p}$ durante el entrenamiento, dejando el vector sin modificaciones durante la inferencia. En las arquitecturas propuestas para este estudio, el *Dropout* se implementó únicamente en la penúltima capa completamente conectada (*fully connected layer*). Esta decisión arquitectónica responde a los hallazgos de Li et al. (2018) [14], quienes demostraron teóricamente que, aunque tanto el *Batch Normalization* como el *Dropout* son herramientas de regularización muy potentes, su utilización conjunta puede degradar el rendimiento de la red. El uso de *Dropout* altera la varianza de las neuronas al cambiar el modelo del estado de entrenamiento al de inferencia. Por el contrario, durante la inferencia, la capa de *Batch Normalization* mantiene la varianza estadística aprendida en el entrenamiento. Esta discrepancia, definida por Li et al. (2018) [14] como “desplazamiento de varianza” (*variance shift*), vuelve inestable al modelo si el *Dropout* se aplica antes de una capa de normalización. Para evitar esta desarmonía de varianzas, se adoptó la estrategia de aplicar la capa de *Dropout* exclusivamente después de la última capa de *Batch Normalization*, un enfoque cuya validez fue confirmada empíricamente en fases de experimentación previas.

4.6. Spatial Pyramid Pooling layer

La última pieza de la arquitectura presente en los primeros dos modelos es la capa de *Spatial Pyramid Pooling*. En términos generales, las capas de agrupamiento (*pooling*) se insertan comúnmente entre las capas convolucionales en las arquitecturas CNN 2D. A diferencia de las capas convolucionales, sus filtros no poseen pesos aprendibles. Su única función es disminuir progresivamente el tamaño espacial de la entrada. Operaciones como el *max-pooling* o *average-pooling* consisten en deslizar un filtro a lo largo de la imagen, reduciendo el tamaño (*downsampling*) de cada porción de profundidad de la entrada de manera independiente.

Las arquitecturas CNN 2D estándar suelen procesar los datos a través de capas convolucionales y luego pasan el último mapa de características (aplanado a un vector 1D) a capas completamente

conectadas (*fully connected layers*). Dado que el tamaño de este vector 1D depende de las dimensiones de la imagen de entrada, los modelos tradicionales exigen que la imagen tenga un tamaño espacial estrictamente predefinido. Para superar esta limitación, He et al. (2014) [15] propusieron la estrategia de *Spatial Pyramid Pooling*. Esta técnica introduce una capa SPP entre la última capa convolucional y la primera completamente conectada. Una capa SPP permite a la red extraer un vector de características de tamaño fijo independientemente de la resolución de la imagen de entrada.

En el trabajo de los autores [10], se decidió implementar esta estrategia con el objetivo de evaluar el rendimiento del modelo ante imágenes satelitales que capturan regiones de tierra de tamaño variables, eliminando la necesidad de preprocesar las imágenes o de modificar la arquitectura del modelo. Asimismo, esto permitió explorar cuál es el tamaño de imagen óptimo para un modelo que cuenta con un número fijo de parámetros.

Una capa de *Spatial Pyramid Pooling* tiene un número predefinido de filtros de *pooling*. Para adaptarse a entradas de tamaño arbitrario, las dimensiones de estos filtros y el salto (*stride*) con el que se deslizan se calculan de forma dinámica. Dado un tensor de entrada 3D con altura H_x y anchura W_x , si se requiere que la salida del filtro tenga una altura H_k y anchura W_k , el paso vertical (s_h) y horizontal (s_w), junto con el tamaño espacial del filtro ($k_h \times k_w$), se rigen por las siguientes ecuaciones:

$$\begin{aligned} s_h &= \lfloor H_x / H_k \rfloor \\ k_h &= \lfloor H_x / H_k \rfloor + (H_x \bmod H_k) \\ s_w &= \lfloor W_x / W_k \rfloor \\ k_w &= \lfloor W_x / W_k \rfloor + (W_x \bmod W_k) \end{aligned}$$

Donde $\lfloor \cdot \rfloor$ representa la función de piso. La salida resultante mantiene la misma profundidad espacial que el tensor de entrada original, pero con las dimensiones espaciales deseadas, permitiendo a la red generar un vector de longitud constante al aplanar y concatenar las salidas de los múltiples filtros piramidales.

4.7. Redes Convolucionales Recurrentes LSTM (Long Short-Term Memory)

Para llegar a los modelos que se uso en [3] basados en Redes Convolucionales Recurrentes LSTM, primero se analizara algunos conceptos clave que fundamentan la idea del uso de este tipo de modelos en el ámbito de la predicción de deforestación.

4.7.1. Redes Neuronales Recurrentes

A diferencia de los modelos CNN y 3D-CNN presentados anteriormente, los cuales son *feed-forward* (la señal fluye en una sola dirección, desde la capa de entrada hasta la salida, sin retroalimentación entre capas), una Red Neuronal Recurrente (RNN) introduce ciclos en la red: la salida de la capa oculta en el paso $t - 1$ se concatena con la nueva imagen de entrada en t y se retroalimenta a la misma capa, dotando al modelo de una “memoria interna” que acumula información de todas las imágenes procesadas hasta ese momento. La motivación detrás de este enfoque es que la deforestación no es necesariamente un evento súbito, sino un proceso continuo, por lo que la estructura temporal de las imágenes satelitales podría aportar información relevante para predecir si un área se deforestará en el año siguiente.

La propagación hacia adelante de una RNN “vanilla” con una sola capa oculta está gobernada por la siguiente ecuación:

$$s_t = \phi(W_s \times s_{t-1} + U_x \times x_t + b)$$

Donde s_t es el estado oculto (la “memoria” acumulada) en el paso t , x_t es la imagen de entrada en ese mismo paso, W_s y U_x son las matrices de pesos que ponderan, respectivamente, la memoria pasada s_{t-1} y la nueva entrada x_t , b es el vector de sesgo (bias), y ϕ es una función de activación no lineal (típicamente la tangente hiperbólica). El estado inicial se define como $s_0 = 0$. En otras palabras, el nuevo estado oculto es una combinación ponderada de lo que la red ya sabía y de la nueva información recibida, la cual se reutiliza en cada paso temporal con los mismos parámetros W_s , U_x y b ; característica que da nombre a este tipo de red.

Como se ilustra en el recuadro rojo de la Figura 4 B, existen dos arquitecturas principales según el objetivo de predicción:

- **Many-to-many:** se genera una predicción $\hat{Y}_k$ en cada paso temporal, y la pérdida total es la suma de las pérdidas individuales: $L_{tot} = \sum_k BCEloss(\hat{Y}_{k+1}, Y_{k+1})$.
- **Many-to-one:** solo se utiliza el último estado oculto (el más informado) para generar una única predicción final, reportando únicamente la pérdida final $L = BCEloss(\hat{Y}_{t+1}, Y_{t+1})$.

La Figura 4 resume esta comparación: el panel A muestra la arquitectura *feed-forward* (CNN/3D-CNN), mientras que el panel B muestra la arquitectura recurrente, donde el recuadro rojo distingue entre calcular la pérdida en cada paso temporal (*many-to-many*) o solo al final de la secuencia (*many-to-one*).

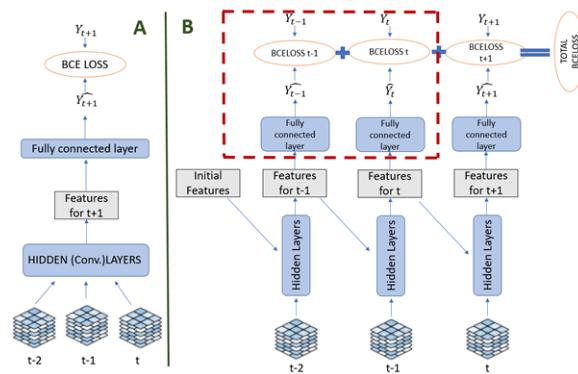


Figura 4: A) Modelo CNN con arquitectura *feed-forward*: las señales de las imágenes viajan en una sola dirección, desde la capa de entrada hasta la capa de salida. B) Red Neuronal Recurrente “vanilla” con aprendizaje recurrente: la salida de la capa oculta se retroalimenta, se concatena con la nueva imagen y se reenvía a la misma capa. Cuando la salida de la capa también se reenvía a la siguiente en cada paso temporal, el modelo tiene arquitectura *many-to-many*. Cuando la capa oculta solo reenvía su salida en la última iteración temporal, la arquitectura es *many-to-one*.

4.7.2. Celdas LSTM

Al procesar datos con secuencias temporales largas, las Redes Neuronales Recurrentes (RNN) estándar a menudo enfrentan el problema de gradientes explosivos (*exploding gradients*) o gra-

dientes desvanecientes (*vanishing gradients*) durante el proceso de retropropagación (*backpropagation*). Mientras que los gradientes explosivos pueden solucionarse mediante el truncamiento, el desvanecimiento de gradientes es un desafío mayor. Para superar este obstáculo, Hochreiter y Schmidhuber (1997) [16] propusieron la arquitectura de memoria a corto y largo plazo (LSTM, por sus siglas en inglés).

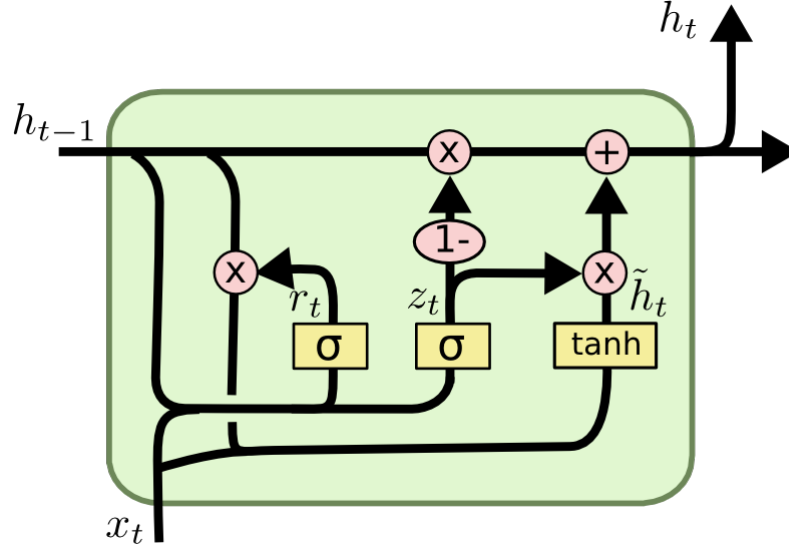


Figura 5: Representación del flujo de información en la arquitectura de una celda LSTM. A diferencia de una RNN tradicional, la entrada actual (x_t) y la salida anterior (h_{t-1}) no solo generan un nuevo vector de características (g_t), sino que también alimentan tres compuertas con activación sigmoide que regulan el paso de la información: la compuerta de olvido (f_t) decide qué proporción de la memoria pasada (c_{t-1}) será bloqueada; la compuerta de entrada (i_t) controla cuánto del nuevo vector aprendido (g_t) se acumula en el estado interno de la celda (c_t); y finalmente, la compuerta de salida (o_t) regula la memoria actualizada para emitir la salida del modelo. Tomado de [17]

Siguiendo la formulación formal expuesta por Goodfellow et al. (2016) [18], la dinámica de una celda LSTM está gobernada por las siguientes ecuaciones:

$$\begin{aligned} f_t &= \sigma(W_f h_{t-1} + U_f x_t + b_f) \\ i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i) \\ o_t &= \sigma(W_o h_{t-1} + U_o x_t + b_o) \\ g_t &= \tanh(W_g h_{t-1} + U_g x_t + b_g) \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

Donde: $x_t \in \mathbb{R}^d$: es el vector de entrada a la unidad LSTM en el tiempo t .

$h_t \in \mathbb{R}^h$: es el vector de salida o estado oculto (*hidden state*).

$c_t \in \mathbb{R}^h$: representa el vector del estado interno de la celda (*internal state*).

$g_t \in \mathbb{R}^h$: es el vector de activación de entrada, que aprende nuevas características combinando la entrada actual y la salida anterior de manera similar a una RNN tradicional.

$f_t, i_t, o_t \in \mathbb{R}^h$: son los vectores de activación de las compuertas de olvido (*forget gate*), entrada (*input gate*) y salida (*output gate*), respectivamente.

$W \in \mathbb{R}^{h \times h}$, $U \in \mathbb{R}^{h \times d}$ y $b \in \mathbb{R}^h$: son las matrices de pesos y los vectores de sesgo (*bias*) que el modelo aprende durante el entrenamiento.

σ y $\tanh$: denotan las funciones de activación no lineal sigmoide y tangente hiperbólica.

$\odot$: representa el producto de Hadamard (multiplicación elemento por elemento).

El aspecto clave de la celda LSTM es su capacidad para decidir qué información mantener, actualizar o descartar, utilizando sus tres compuertas equipadas con activaciones sigmoides: La compuerta de olvido (*forget gate*) (f_t) decide qué proporción de la información del estado anterior de la celda (c_{t-1}) será bloqueada o “olvidada”. La compuerta de entrada (*input gate*) (i_t) controla qué cantidad de la nueva información aprendida (g_t) será acumulada en el estado interno actual de la celda (c_t). La compuerta de salida (*output gate*) (o_t) regula la memoria actualizada (c_t), previamente comprimida por una función tangente hiperbólica, para generar el estado oculto final (h_t) que se emitirá en ese instante temporal. Esta estructura de compuertas permite que la red estabilice el flujo de los gradientes, lo que hace posible apilar temporalmente múltiples capas LSTM para construir arquitecturas más profundas capaces de aprender dependencias temporales a largo plazo.

4.8. Arquitecturas Utilizadas

4.8.1. Arquitectura 2DCNN

El primer modelo consiste en una Red Neuronal Convolutiva 2D compuesta por cuatro capas convolucionales 2D (2D Conv), una capa de *Spatial Pyramid Pooling* o SPP y dos capas completamente conectadas (FC) separadas por una capa de *Dropout* (DO).

Para un punto de datos j , la entrada de la red es un tensor 3D denotado como SX_t^j , con dimensiones $\mathbb{R}^{7 \times (2r+1) \times (2r+1)}$. Este tensor de entrada se construye apilando elementos a lo largo del eje de canales de la siguiente manera: Se utiliza un tensor estático 3D $S_j \in \mathbb{R}^{2 \times (2r+1) \times (2r+1)}$. Se apila con un tensor 3D $X_t^j \in \mathbb{R}^{2 \times (2r+1) \times (2r+1)}$ proveniente de la serie temporal de tensores 3D, denotada como $\{X_k^j\}_{k=t-2}^t$. Gracias a la incorporación de la capa SPP, el modelo es capaz de analizar tensores 3D de cualquier tamaño espacial, el cual está regulado por el parámetro r . Los filtros de cada capa convolutiva son activados mediante la función ReLU. Dichas activaciones se normalizan posteriormente con una capa de *Batch Normalization* 2D (2DBN). El flujo de propagación entre las capas convolucionales sigue la secuencia: 2DConv $\rightarrow$ ReLU $\rightarrow$ 2DBN $\rightarrow$ 2DConv. Las activaciones normalizadas de la última capa convolutiva 2D son reducidas espacialmente (*downsampled*) por la capa SPP y transmitidas a la primera capa completamente conectada (FC). Esta primera capa FC genera un vector de características de tamaño 100, el cual se activa nuevamente con ReLU y se normaliza con una capa BN 1D. Como sugieren Li et al. (2018) [14], la capa de *Dropout* se aplica únicamente después de esta última capa de normalización (BN). La capa FC final procesa el vector y devuelve un valor escalar. Este valor se comprime mediante una función sigmoide (σ), lo que produce una salida en el rango $[0, 1]$. La etiqueta predicha resultante, $\hat{Y}_{t+1}^j$ (donde $j \in J_t$), indica la confianza del modelo en observar deforestación en la ubicación correspondiente al píxel central espacial (con coordenadas espaciales $(r+1, r+1)$) del tensor de entrada durante el año siguiente.

Esta arquitectura fue diseñada de manera flexible, permitiendo la variación de múltiples parámetros del modelo: El tamaño espacial, el salto (*stride*) y el relleno (*padding*) de los filtros en cualquiera de las capas convolucionales 2D. El número de filtros por capa convolutiva. Los parámetros de la capa SPP, incluyendo el número de filtros de agrupamiento (n) y el tamaño

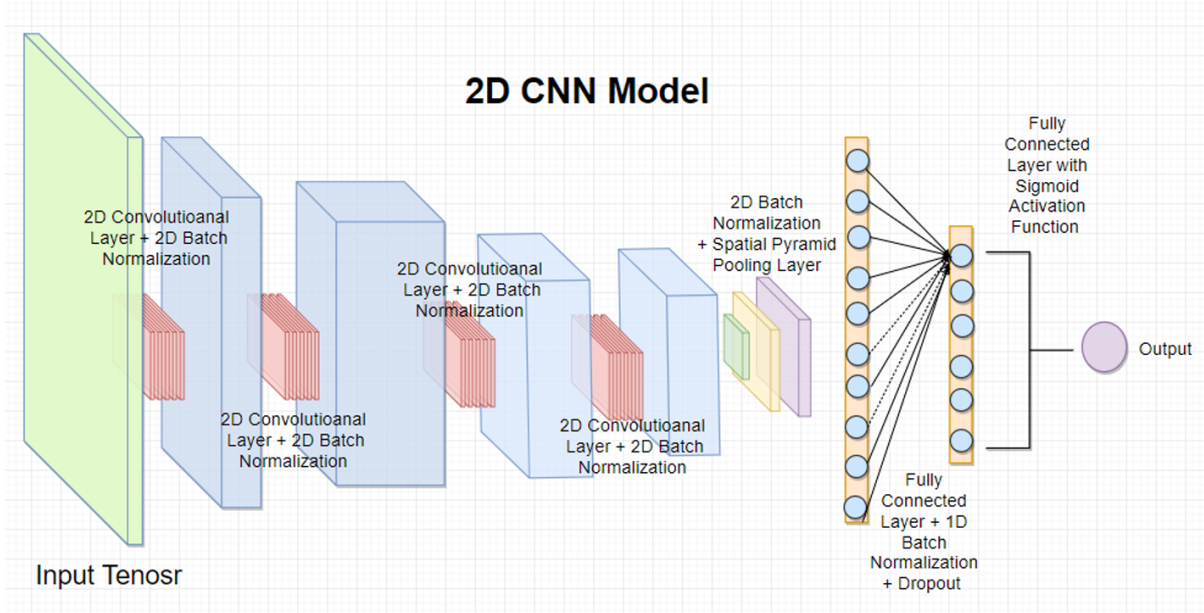


Figura 6: El flujo de propagación completo del Modelo 1 se resume de la siguiente manera: $\mathbf{S}\mathbf{X}_t^j \in \mathbb{R}^{7 \times (2r+1) \times (2r+1)} \rightarrow 4 \times (2\text{DConv} \rightarrow \text{ReLU} \rightarrow 2\text{DBN}) \rightarrow \text{SPP}(n, k) \rightarrow \text{FC}(\text{spp}, 100) \rightarrow \text{ReLU} \rightarrow 1\text{DBN} \rightarrow \text{DO}(p) \rightarrow \text{FC}(100, 1) \rightarrow \sigma \rightarrow \hat{Y}_{t+1}^j$. Tomado de [10]

espacial de estos filtros ($k \in \mathbb{R}^{2,n}$). Variar estos parámetros altera el tamaño de la primera capa FC, ya que este depende directamente de la salida de la capa SPP y la tasa de Dropout (p). Un resumen de la arquitectura se encuentra en la figura 6

4.8.2. Arquitectura 3DCNN

Mientras que el modelo base (2D CNN) aprovecha eficientemente las características espaciales presentes en las imágenes satelitales, carece de la capacidad de utilizar directamente el dominio temporal para la caracterización y predicción de la ventana, ya que analiza únicamente un elemento de la serie temporal. Nuestro segundo modelo, la Red Neuronal Convolutiva 3D (3D CNN), supera esta limitación. Esto se logra mediante una arquitectura compuesta por dos ramas independientes que procesan diferentes partes de los datos y luego convergen para generar la predicción final.

Para el punto de datos j , el modelo toma como entrada dos componentes separados: El tensor estático $S_j \in \mathbb{R}^{2 \times (2r+1) \times (2r+1)}$. La serie temporal completa de tensores no estáticos $\{X_{t-2}^j, X_{t-1}^j, X_t^j\}$, donde cada elemento pertenece a $\mathbb{R}^{5 \times (2r+1) \times (2r+1)}$. Estos dos componentes se dirigen a ramas diferentes para la extracción inicial de características de alto nivel: La serie temporal se apila por su dominio de tiempo formando un tensor 4D $X^j \in \mathbb{R}^{5 \times 3 \times (2r+1) \times (2r+1)}$ (donde las dimensiones corresponden a canales, tiempo, altura y anchura, respectivamente). Esta rama convoluciona sobre las tres últimas dimensiones (tiempo, altura y anchura) a través de dos capas convolucionales 3D secuenciales. Dado el límite del dominio temporal (tamaño 3), los filtros 4D tienen la forma $\in \mathbb{R}^{\text{canales}, 2, k_h, k_w}$ (el tamaño del dominio temporal del filtro solo puede ser 2). Los filtros se deslizan con un salto (*stride*) de 1 y sin aplicar relleno (*padding*) al tensor de entrada. La salida de esta rama es un tensor 3D de características de alto nivel que ha perdido su dimensión temporal, denotado como $Z_x^j \in \mathbb{R}^{c_1, h, w}$.

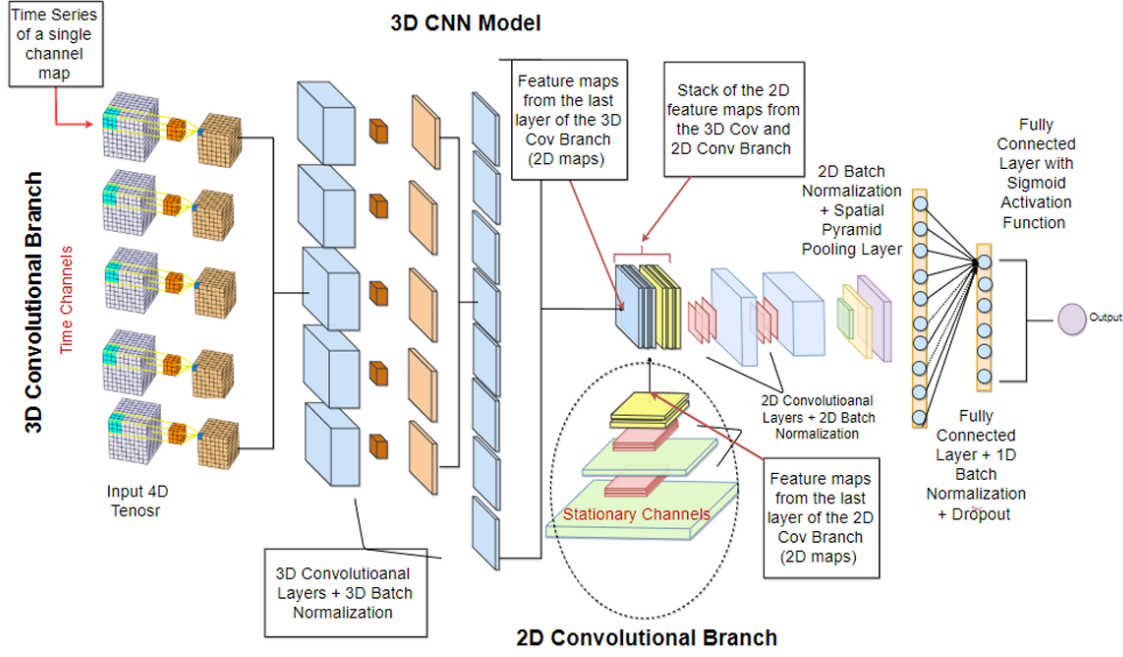


Figura 7: El flujo completo de propagación de este modelo se resume de la siguiente manera:

$$\begin{aligned}
 \mathbf{X}_t^j &\in \mathbb{R}^{5 \times 3 \times (2r+1) \times (2r+1)} \rightarrow 2 \times (3DConv \rightarrow ReLU \rightarrow 3DBN) \rightarrow Z_x^j \in \mathbb{R}^{c_1, h, w} \\
 \mathbf{S}_j &\in \mathbb{R}^{2 \times (2r+1) \times (2r+1)} \rightarrow 2 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow Z_s^j \in \mathbb{R}^{c_2, h, w} \\
 \mathbf{Z}_t^j &\in \mathbb{R}^{(c_1+c_2), h, w} \rightarrow 2 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow SPP(n, k) \rightarrow FC(spp, 100) \rightarrow ReLU \rightarrow \\
 &1DBN \rightarrow DO(p) \rightarrow FC(100, 1) \rightarrow \sigma \rightarrow \hat{Y}_{t+1}^j. \text{ Tomado de [10]}
 \end{aligned}$$

Luego se tiene una rama convolucional 2D donde el tensor estático S_j se procesa a lo largo del dominio espacial mediante dos capas convolucionales 2D secuenciales. Los filtros convolucionales de ambas ramas se configuran con el mismo tamaño espacial, lo que garantiza que los tensores resultantes tengan las mismas dimensiones espaciales ($h \times w$). El resultado de esta rama es el tensor 3D de características de alto nivel $Z_s^j \in \mathbb{R}^{c_2, h, w}$.

Los tensores resultantes de ambas ramas (Z_x^j y Z_s^j) se apilan a lo largo de su tercera dimensión para formar el tensor unificado $Z^j \in \mathbb{R}^{(c_1+c_2), h, w}$. Este tensor se propaga por el resto de la red, pasando por dos capas convolucionales 2D adicionales, una capa SPP y dos capas completamente conectadas (FC) separadas por una capa de *Dropout* (DO). La salida final es comprimida por una función sigmoide (σ), retornando un valor $\hat{Y}_{t+1}^j$ que indica la confianza del modelo en observar deforestación en la ubicación objetivo durante el año $t + 1$.

Al igual que en el modelo anterior, se aplica normalización (2DBN o 3DBN) entre las capas convolucionales después de la función de activación ReLU. El número de filtros, su tamaño espacial y los parámetros de las capas SPP y DO se configuraron como parámetros libres para permitir la experimentación, manteniendo la flexibilidad de analizar tensores de cualquier tamaño espacial. Un resumen de la arquitectura se encuentra en la figura 7

4.8.3. Arquitecturas de los Modelos 3 y 4 Redes Neuronales Recurrentes ConvLSTM

Aunque el Modelo 2 (3D CNN) es capaz de utilizar las tres dimensiones de los datos de entrada, su aprendizaje se basa en una arquitectura (*feed-forward*). Dado que el aprendizaje recurrente suele ser un proceso muy potente para el manejo de datos secuenciales, los Modelos 3 y 4 fueron

diseñados para aprender la estructura temporal de los datos de entrada utilizando una o varias celdas *Convolutional Long Short-Term Memory* (ConvLSTM).

Ambos modelos comparten una arquitectura similar a la de la red 3D CNN, dividiendo el procesamiento en dos ramas independientes que luego convergen. Para un punto de datos j , reciben como entrada el tensor estático $S_j \in \mathbb{R}^{2 \times (2r+1) \times (2r+1)}$ y la serie temporal $\{X_{t-2}^j, X_{t-1}^j, X_t^j\}$, donde cada elemento pertenece a $\mathbb{R}^{5 \times (2r+1) \times (2r+1)}$. La serie temporal se propaga a través de la rama temporal (compuesta por un *Encoder* y una sub-rama ConvLSTM), mientras que el tensor estático se procesa en la rama 2D CNN *Static*. Cada rama devuelve un tensor 3D de características de alto nivel (Z_x^j y Z_s^j , respectivamente), los cuales se apilan a lo largo de su eje de canales para formar el tensor conjunto Z^j . La rama temporal difiere del modelo 3D CNN al procesar los datos en dos etapas: El *Encoder* codifica la serie temporal de tensores 3D originales en una nueva serie temporal de características de alto nivel $\{\tilde{X}_k^j\}_{k=t-2}^t$. Para lograrlo, alimenta cada tensor X_k^j a través de las mismas tres capas convolucionales 2D secuenciales. En la rama ConvLSTM RNN la serie temporal codificada es manejada recurrentemente. En el Modelo 3, la serie pasa por una única celda ConvLSTM. El Modelo 4 se diferencia por ser “profundo” (*deep*), pasando la serie temporal a través de una pila de d celdas ConvLSTM. En ambos casos, tras la última iteración, la rama devuelve la salida temporal más reciente de la celda más profunda en forma de un tensor 3D, Z_x^j , el cual almacena la “memoria” espacio-temporal de la entrada. La rama Estática (2D CNN *Static*) es idéntica en los Modelos 3 y 4, y muy similar a la del Modelo 2. El tensor S_j pasa por tres capas convolucionales 2D secuenciales para extraer características de alto nivel. Los filtros convolucionales están configurados para que el tensor de salida, $Z_s^j \in \mathbb{R}^{c_3, h_2, w_2}$, coincida exactamente con el tamaño espacial generado por la sub-rama ConvLSTM. Red Conjunta (2D CNN Joint) y Flexibilidad Los tensores Z_x^j y Z_s^j se concatenan formando $Z^j \in \mathbb{R}^{(c_2+c_3), h_2, w_2}$. Este tensor unificado se propaga por el resto de la red, pasando por tres capas convolucionales 2D, una capa SPP, y dos capas FC separadas por un *Dropout* (DO). Finalmente, la última capa FC utiliza una función sigmoide (σ) para retornar la predicción $\hat{Y}_{t+1}^j \in [0, 1]$. El diseño mantiene una alta flexibilidad, permitiendo ajustar hiperparámetros como el tamaño espacial, el número de filtros, la profundidad d de las celdas ConvLSTM, la tasa de *Dropout* p , y la capacidad de procesar tensores de cualquier tamaño espacial regulado por r .

El Resumen de la Arquitectura del Modelo 4 con $d = 2$ esta ilustrado en la figura

5. Resultados

En el trabajo original, la evaluación de los modelos se dividió en una comparación general de las distintas arquitecturas y un análisis detallado del desempeño predictivo a futuro. A continuación, se resumen los principales hallazgos

5.1. Comparación general de las arquitecturas

Los resultados iniciales demostraron que los mejores modelos de cada clase predijeron la deforestación en el conjunto de prueba de la región de Madre de Dios (año 2018) con precisiones similares, logrando un área bajo la curva (ROC-AUC) de entre 0.935 y 0.944. El modelo con el desempeño más alto fue la red convolucional tridimensional (3D CNN) con un AUC de 0.944. Le siguieron la arquitectura ConvLSTM profunda (AUC = 0.938), la ConvLSTM estándar (AUC = 0.937) y finalmente la 2D CNN (AUC = 0.935). Estos resultados sugieren que los modelos capaces de manejar de forma implícita la dimensión temporal junto con la espacial (como la 3D CNN y las ConvLSTM) son mejores para capturar la evolución de los patrones espacio-temporales de

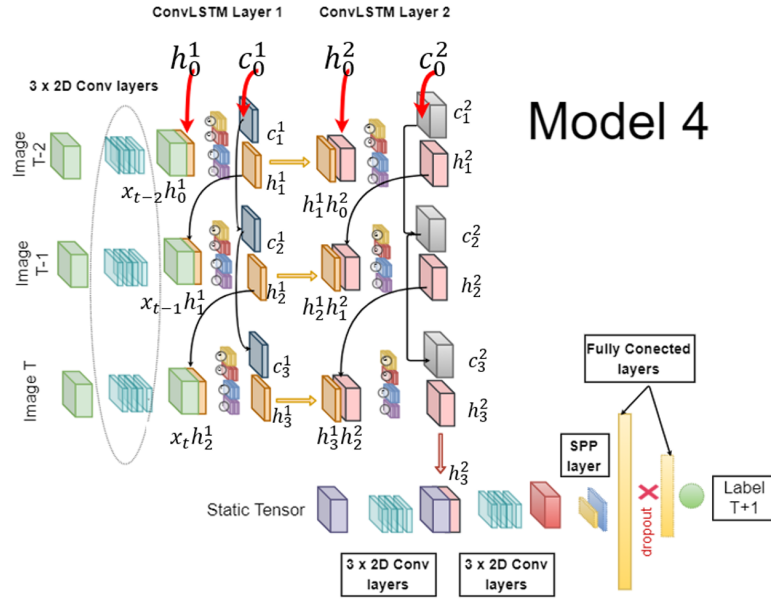


Figura 8: El flujo de procesamiento para el Modelo 4 se resume con las siguientes ecuaciones:

Encoder: $X_k^j, k = t - 2^t \rightarrow \{\tilde{X}_k^j, k = t - 2^t$

$\mathbf{X}_t^j \in \mathbb{R}^{5 \times (2r+1) \times (2r+1)} \rightarrow 3 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow \tilde{X}_t^j \in \mathbb{R}^{c_1 \times h_1 \times w_1}$

$\mathbf{X}_{t-1}^j \in \mathbb{R}^{5 \times (2r+1) \times (2r+1)} \rightarrow 3 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow \tilde{X}_{t-1}^j \in \mathbb{R}^{c_1 \times h_1 \times w_1}$

$\mathbf{X}_{t-2}^j \in \mathbb{R}^{5 \times (2r+1) \times (2r+1)} \rightarrow 3 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow \tilde{X}_{t-2}^j \in \mathbb{R}^{c_1 \times h_1 \times w_1}$

Conv LSTM RNN: $\{\tilde{X}_k^j\}, k = t - 2^t \rightarrow 2 \times (ConvLSTM) \rightarrow Z_x^j \in \mathbb{R}^{c_2, h_2, w_2}$

2D CNN Static:

$\mathbf{S}_j \in \mathbb{R}^{2 \times (2r+1) \times (2r+1)} \rightarrow 3 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow Z_s^j \in \mathbb{R}^{c_3, h_2, w_2}$

2D CNN Joint:

$\mathbf{Z}^j \in \mathbb{R}^{(c_2+c_3), h_2, w_2} \rightarrow 3 \times (2DConv \rightarrow ReLU \rightarrow 2DBN) \rightarrow SPP(n, k) \rightarrow FC(spp, 100) \rightarrow ReLU \rightarrow 1DBN \rightarrow DO(p) \rightarrow FC(100, 1) \rightarrow \sigma \rightarrow \hat{Y}^j t + 1$

la deforestación. A pesar del buen rendimiento de los modelos recurrentes (ConvLSTM), resultaron poco competitivos en términos de tiempo de entrenamiento debido a la dificultad para paralelizar sus procesos. Por esta razón, los autores decidieron continuar únicamente con las arquitecturas 2D CNN y 3D CNN.

5.2. Desempeño predictivo y pronóstico a futuro

Al afinar los hiperparámetros y extender el entrenamiento, las redes 2D CNN y 3D CNN superaron ampliamente al modelo base de *Random Forest*. Se destacaron los siguientes comportamientos:

Las predicciones en la región de Junín fueron consistentemente menos precisas que en Madre de Dios. Esto se atribuye a que Junín es un entorno inherentemente más complejo de predecir debido a una mayor cobertura nubosa, diversidad de tipos de bosque, topología y fragmentación del ecosistema.

Al evaluar la capacidad del modelo para pronosticar eventos a un año en el futuro (evaluado con etiquetas del año 2020), la 3D CNN fue el modelo más exacto para Madre de Dios, alcanzando un F1-Score de 0.715. Curiosamente, para la región de Junín, el mejor pronóstico lo obtuvo la 2D CNN ($F1 = 0.623$). Los autores sugieren que al procesar los años de manera individual (en lugar de apilar los datos temporalmente como la 3D CNN), la 2D CNN es más resistente al ruido provocado por la nubosidad persistente.

Un hallazgo muy relevante fue que el modelo 3D CNN entrenado exclusivamente con datos de Madre de Dios obtuvo los mejores resultados al realizar inferencia sobre Junín. Esto demuestra que los modelos tienen la capacidad de transferir su aprendizaje espacial a nuevas regiones y que la calidad de los datos es, en ocasiones, más importante que el simple volumen o la diversidad geográfica de los mismos. Ajustes de variables: Se determinó que el tamaño óptimo de la ventana espacial de entrada es de aproximadamente 1 a 1.5 kilómetros cuadrados (35x35 a 50x50 píxeles), punto en el cual el modelo aprovecha el contexto local sin perder el foco en el píxel central. Además, se comprobó que la capa topográfica de elevación no mejoró el rendimiento, por lo que fue descartada.

Más allá de las métricas cuantitativas, la inspección visual de los pronósticos de riesgo para el año 2021 evidenció que la red neuronal logró aprender a discernir de manera automática entre diferentes motores de deforestación. El modelo asignó niveles de riesgo muy altos a los bordes de fronteras antrópicas activas, tales como zonas de expansión agrícola comercial, minas de oro no autorizadas y nuevas rutas de acceso (carreteras). En marcado contraste, los eventos recientes de pérdida de bosque causados por dinámicas naturales aisladas, como deslizamientos de tierra remotos, fueron clasificados con un riesgo muy bajo de seguir expandiéndose. Esto demuestra una capacidad de discriminación notable, identificando qué frentes tienen probabilidades reales de propagación progresiva.

6. Contexto Colombiano

El estudio de Ball et al. (2022) [3] se centra en las regiones de Madre de Dios y Junín en la Amazonía peruana, donde la deforestación es impulsada por la minería de oro legal e ilegal, concesiones madereras, expansión agrícola, ganadería y vías de acceso no oficiales. Este panorama comparte profundas similitudes con los boletines de Alertas Tempranas de Deforestación del

IDEAM en Colombia [1]. Los núcleos de deforestación activos en Colombia (como los ubicados en el arco amazónico: Guaviare, Caquetá, Meta y Putumayo) presentan motores de pérdida de bosque análogos, destacando la praderización (acaparamiento de tierras para ganadería), los cultivos de uso ilícito, la minería criminal y la apertura de vías informales [1]. Dado que gran parte de la infraestructura vial en regiones de bosques tropicales no está cartografiada oficialmente y es altamente dinámico. Es por esto que las arquitecturas propuestas en Ball et al. (2022) son propicias para el pronóstico de la deforestación, por las ventajas discutidas anteriormente.

6.1. Adaptación del Modelo

Si bien el modelo 3D CNN original posee alta capacidad predictiva, su implementación íntegra exige recursos de procesamiento intensivos. Para abordar los límites computacionales específicos de este proyecto, se diseñó la MDFNet, una adaptación a menor escala que conserva el razonamiento bimodal estático-temporal de Ball (2022).

La MDFNet se configuró para recibir parches espaciales de tamaño 17×17 centradas alrededor del pixel a predecir y consta de las siguientes etapas:

La rama estática procesa características, representadas como un tensor de dimensiones $2 \times 17 \times 17$. Está compuesta por dos bloques idénticos que aplican operaciones Conv2D (con filtros de 3×3), BN2D y activación ReLU. Esta rama proyecta los 2 canales iniciales a un espacio de características $Z_s \in \mathbb{R}^{16 \times 17 \times 17}$.

La rama temporal analiza la secuencia histórica a través de un tensor de dimensiones $8 \times 5 \times 17 \times 17$ (con 8 canales y una ventana temporal de 5 años). Primero, emplea un bloque que extrae las características espacio-temporales con una Conv3D (kernel $3 \times 3 \times 3$), BN3D ReLU para aumentar los canales a 16. Luego, efectúa una agregación temporal utilizando otra Conv3D con kernel asimétrico de $(3 \times 1 \times 1)$. Esto permite comprimir y remover la dimensión temporal, arrojando un mapa de características 2D consolidado $Z_t \in \mathbb{R}^{16 \times 17 \times 17}$.

Siguiendo la lógica de Ball (2022), las representaciones Z_s y Z_t se concatenan a lo largo del eje de canales, resultando en un volumen de dimensiones $32 \times 17 \times 17$). Este tensor unificado pasa por capas Conv2D. Para lograr la reducción global previa a la clasificación se aplica una capa de SPP (resoluciones 1×1 , 2×2 y 4×4 para crear un vector denso de 336 posiciones). Finalmente, el vector latente pasa por un clasificador de redes neuronales FC, aplicando regularización por DO con probabilidad de 0,2, y la salida es pasada por una función Sigmoide para obtener un valor entre (0, 1) que se puede interpretar como la probabilidad de que el pixel sea deforestado en el año siguiente.

Esta arquitectura esta resumida de la siguiente manera:

$$\begin{aligned} \mathbf{X}^j &\in \mathbb{R}^{8 \times 5 \times 17 \times 17} \rightarrow 2 \times (3DConv \rightarrow 3DBN \rightarrow ReLU) \rightarrow Z_t^j \in \mathbb{R}^{16 \times 17 \times 17} \\ \mathbf{S}^j &\in \mathbb{R}^{2 \times 17 \times 17} \rightarrow 2 \times (2DConv \rightarrow 2DBN \rightarrow ReLU) \rightarrow Z_s^j \in \mathbb{R}^{16 \times 17 \times 17} \\ \mathbf{Z}^j &\in \mathbb{R}^{32 \times 17 \times 17} \rightarrow 2 \times (2DConv \rightarrow 2DBN \rightarrow ReLU) \rightarrow SPP(1 \times 1, 2 \times 2, 4 \times 4) \rightarrow FC(336, 16) \rightarrow \\ &ReLU \rightarrow DO(0,2) \rightarrow FC(16, 1) \rightarrow \sigma \rightarrow \hat{Y}_{t+1}^j \end{aligned}$$

7. Experimentos y Configuración Experimental

Para validar la capacidad predictiva del modelo de aprendizaje profundo propuesto (*MDFNet*) en la estimación de los patrones espaciales de deforestación en el municipio de Mapiripán (Meta), se diseñó e implementó un flujo de trabajo experimental riguroso estructurado en Python mediante

el marco de trabajo *PyTorch*.

7.1. Área de Estudio y Dataset de Entrada

El área de interés abarca una ventana espacial centrada en las coordenadas geográficas $3,30^{\circ}$ N y $-71,55^{\circ}$ O, correspondiente a una extensión de $15 \times 15 \text{ km}^2$ más un margen espacial de seguridad (*padding*) de 500 m. Los insumos raster fueron extraídos del conjunto de datos *Global Forest Change* (GFC) [?] cubriendo las versiones v1.10 a v1.13 (2022–2025).

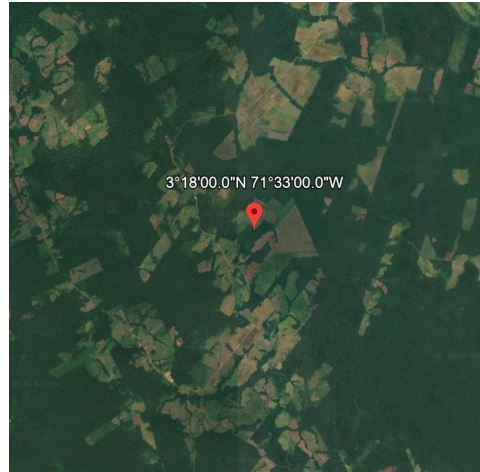


Figura 9: Ubicación del área de estudio en el municipio de Mapiripán (Meta). El recuadro destaca la región de $15 \times 15 \text{ km}$ seleccionada para el análisis de deforestación. Imágenes ©2026 Maxar Technologies, Datos del mapa ©2026 Google.

La arquitectura de datos diferencia dos tipos de covariables de entrada procesadas de manera multicanal:

- **Variables Estáticas:** Cobertura arbórea del año 2000 (*tc2000*), ganancia forestal acumulada (*gain*), máscara de datos válidos (*mask*) e historial consolidado de pérdida de bosque (*loss*).
- **Variables Temporales:** Imágenes espectrales infrarrojas correspondientes al último sensor disponible (*last*) para la secuencia multitemporal.

7.2. Estrategia de Muestreo y Experimentos Diseñados

Debido a que la deforestación es un evento espacialmente escaso, el conjunto de datos exhibe un severo desbalance de clases (donde los píxeles sin deforestación superan ampliamente a los píxeles deforestados). Para mitigar este sesgo y evaluar la sensibilidad del modelo ante la proporción de muestras, los datos se dividieron de forma aleatoria fija (semilla *seed=42*) en tres subconjuntos independientes:

- **Entrenamiento (*Train*):** 60 % del total de muestras.
- **Validación (*Validation*):** 20 % del total de muestras.
- **Prueba (*Test*):** 20 % del total de muestras.

Se diseñaron tres experimentos principales comparativos ajustando la estrategia de submuestreo (*undersampling*) sobre la clase negativa en el conjunto de entrenamiento:

1. **Baseline:** Mantiene la distribución natural de muestras sin aplicar submuestreo.
2. **Undersampling 1:15:** Submuestreo de la clase negativa ajustado a una proporción de 15 píxeles no deforestados por cada píxel deforestado.
3. **Undersampling 1:10:** Submuestreo de la clase negativa ajustado a una proporción de 10 píxeles no deforestados por cada píxel deforestado.

7.3. Configuración de Entrenamiento e Hiperparámetros

El entrenamiento de *MDFNet* se ejecutó utilizando la función de pérdida de entropía cruzada binaria con logits (*BCEWithLogitsLoss*), dada por:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\sigma(\hat{y}_i)) + (1 - y_i) \cdot \log(1 - \sigma(\hat{y}_i))] \quad (1)$$

donde $y_i \in \{0, 1\}$ representa la etiqueta real, $\hat{y}_i$ denota la salida lineal (*logits*) emitida por la red y $\sigma(\cdot)$ es la función sigmoide.

Los parámetros de optimización fijados para todos los experimentos se detallan a continuación:

- **Optimizador:** Adam con una tasa de aprendizaje inicial (η) de 1×10^{-3} .
- **Programador de Tasa de Aprendizaje:** *ReduceLROnPlateau*, atenuando el valor de η por un factor de 0,5 si la pérdida de validación no disminuye durante 3 épocas consecutivas, con un límite inferior de $\eta_{\min} = 1 \times 10^{-6}$.
- **Tamaño de Lote (*Batch Size*):** 64 muestras.
- **Número Máximo de Épocas:** 200.
- **Parada Temprana (*Early Stopping*):** Configurada con una paciencia de 25 épocas monitoreando el área bajo la curva ROC (*ROC-AUC*) en el conjunto de validación, con un incremento mínimo requerido $\Delta = 1 \times 10^{-4}$.

El modelo correspondiente a la época que alcanzó el mayor valor de *ROC-AUC* en validación fue almacenado como el punto de control óptimo (*checkpoint*) para la fase de evaluación.

7.4. Pipeline de Evaluación y Métricas de Desempeño

Durante la fase de validación se determinó dinámicamente el umbral óptimo de decisión de probabilidad (τ). Posteriormente, en la etapa de prueba e inferencia independiente (*Testing Pipeline*), las probabilidades predichas $p_i = \sigma(\hat{y}_i)$ se binarizaron mediante:

$$\hat{c}_i = \begin{cases} 1, & \text{si } p_i \geq \tau \\ 0, & \text{si } p_i < \tau \end{cases} \quad (2)$$

Para evaluar el desempeño global del modelo sobre el conjunto de test (completamente desacoplado del entrenamiento) se calcularon cinco métricas estandarizadas:

- **Pérdida en Test (*Test Loss*):** Promedio de $\mathcal{L}_{\text{BCE}}$ sobre las muestras de prueba.
- **Precisión (*Precision*):** Proporción de verdaderos positivos respecto a todos los píxeles clasificados como deforestados ($\frac{TP}{TP+FP}$).
- **Exhaustividad / Sensibilidad (*Recall*):** Proporción de píxeles deforestados correctamente identificados ($\frac{TP}{TP+FN}$).
- **Puntuación F1 (*F1-Score*):** Media armónica entre Precisión y Exhaustividad ($2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$).
- **ROC-AUC:** Área bajo la curva de característica operativa del receptor, evaluando la capacidad de discriminación del modelo independientemente del umbral.

8. Resultados

En esta sección se presentan los resultados cuantitativos y cualitativos obtenidos por la arquitectura *MDFNet*. En primer lugar, se realiza un análisis comparativo de los tres escenarios experimentales de submuestreo. Posteriormente, se evalúa a detalle el desempeño del modelo con mejor capacidad de generalización y, finalmente, se exponen los resultados del pronóstico espacial de deforestación para el año 2026 en el área de estudio de Mapiripán (Meta).

8.1. Evaluación Comparativa de Experimentos

La Cuadro 2 resume el desempeño cuantitativo de los tres experimentos evaluados sobre el conjunto de prueba independiente (20 % de los datos), el cual conservó la distribución espacial real del territorio.

Cuadro 2: Desempeño cuantitativo de la arquitectura *MDFNet* en el conjunto de prueba independiente para las diferentes estrategias de muestreo.

Experimento	Época	τ^*	($\mathcal{L}_{\text{BCE}}$)	Precisión	Sensibilidad	F_1 -score	ROC-AUC
<i>Baseline</i>	47	0.60	0.0359	0.8544	0.8547	0.8546	0.9949
<i>Undersampling 1:15</i>	39	0.73	0.0396	0.8230	0.8230	0.8230	0.9932
<i>Undersampling 1:10</i>	39	0.79	0.0416	0.8315	0.8247	0.8281	0.9932

Contrario a la hipótesis inicial sobre la necesidad de balancear artificialmente las clases, la configuración *Baseline* (sin submuestreo) alcanzó el rendimiento superior en todas las métricas evaluadas. Obtuvo el menor valor de pérdida en prueba ($\mathcal{L}_{\text{BCE}} = 0,0359$), una precisión de 85,44 %, una sensibilidad (*recall*) de 85,47 % y un F_1 -score de 0,8546.

Las estrategias de submuestreo (*1:15* y *1:10*) convergieron de manera más temprana (época 39) y requirieron umbrales de decisión mayores ($\tau^* = 0,73$ y $\tau^* = 0,79$, respectivamente), pero experimentaron un leve deterioro en la capacidad de discriminación general, reduciendo el F_1 -score a 0,8230 y 0,8281. Esto sugiere que preservar la variabilidad completa de las muestras negativas durante el entrenamiento permite a la red *MDFNet* aprender representaciones contextuales más robustas del trasfondo no deforestado sin sobreestimar el riesgo de cambio.

8.2. Análisis del Desempeño del Modelo Óptimo

El modelo óptimo correspondió a la configuración *Baseline* en la época de entrenamiento 47, evaluado con un umbral óptimo de probabilidad $\tau^* = 0,60$. La métrica de área bajo la curva ROC

($ROC-AUC = 0,9949$) evidencia una sobresaliente capacidad para clasificar píxeles deforestados y no deforestados a través de distintos umbrales de decisión.

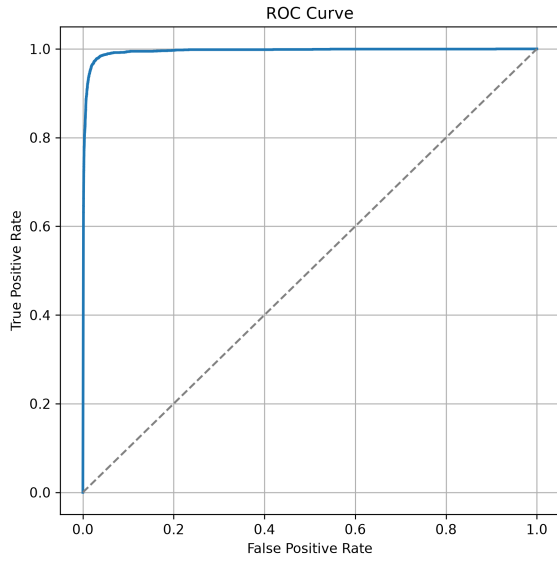


Figura 10: Curva ROC obtenida por el modelo *Baseline* en el conjunto de prueba.

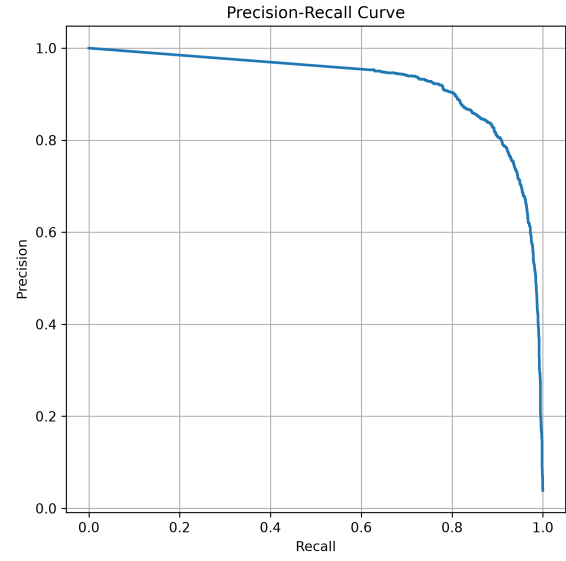


Figura 11: Curva Precisión-Exhaustividad (*PR*) para el modelo *Baseline*.

El equilibrio logrado entre la precisión (85,44 %) y la sensibilidad (85,47 %) demuestra que el modelo no padece de un sesgo hacia falsos positivos ni hacia una omisión alta de eventos de deforestación reales. La Figura 12 ilustra la matriz de confusión resultante en el subconjunto de prueba desacoplado.

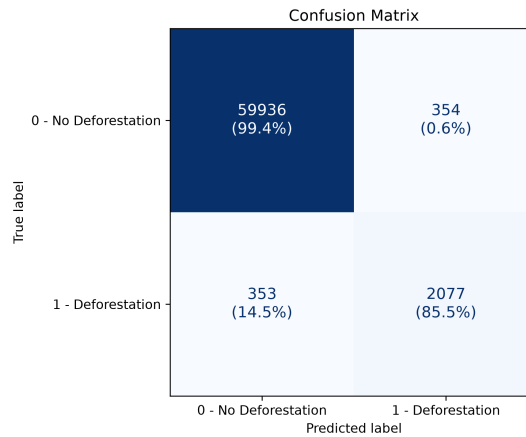


Figura 12: Matriz de confusión del modelo *Baseline* en el conjunto de prueba ($\tau^* = 0,60$).

8.3. Pronóstico Espacial de Deforestación para el Año 2026

Una vez validado el desempeño del modelo *Baseline*, se ejecutó la fase de inferencia prospectiva para predecir los patrones de deforestación en el área de estudio durante el año 2026. La ventana espacial analizada abarca un total de 313,599 píxeles válidos (equivalentes a la grilla de $15 \times 15 \text{ km}^2$ con el área de protección).

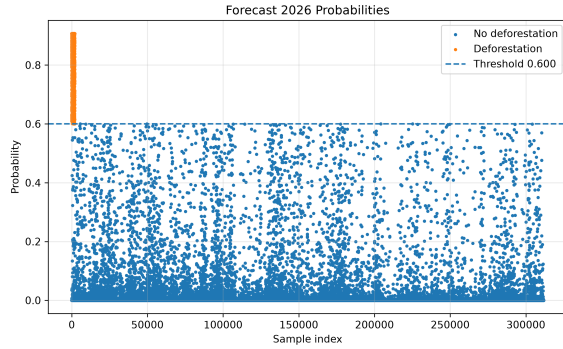


Figura 13: Distribución de probabilidades de deforestación predichas para 2026.

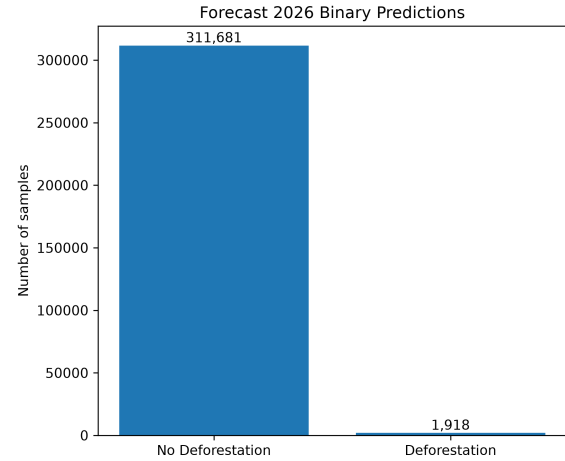


Figura 14: Clasificación binaria de riesgo de deforestación aplicando el umbral $\tau^* = 0,60$.

Aplicando el umbral óptimo de decisión $\tau^* = 0,60$, el modelo pronostica una transformación forestal activa de ****1,918 píxeles**** para el año 2026 (aproximadamente 172,6 hectáreas, considerando la resolución espacial de 30 m por píxel).

Como se observa en la Figura 15, los píxeles predichos con alto riesgo de deforestación se concentran de manera contigua a los frentes de pérdida histórica previa, reproduciendo la dinámica de expansión espacial nucleada característica de este sector del municipio de Mapiripán.

8.4. Evolución de la Función de Pérdida y Tasa de Aprendizaje

La estabilidad en el proceso de optimización es un indicador crítico para descartar problemas de desvanecimiento o explosión del gradiente, así como para garantizar una convergencia suave del modelo. La Figura 16 ilustra la trayectoria de la pérdida de entropía cruzada binaria ($\mathcal{L}_{BCE}$) tanto en el subconjunto de entrenamiento como en el de validación a lo largo de las épocas de entrenamiento.

Como se observa en la Figura 16, durante las primeras 15 épocas la función de pérdida disminuye aceleradamente, pasando de valores superiores a 0,20 a niveles por debajo de 0,05. A partir de la época 30, la pérdida de validación entra en una fase de asíncrona estabilización, donde continúa decreciendo de manera marginal hasta alcanzar su punto de mínima divergencia alrededor de la época 47. La estrecha brecha sostenida entre la curva de entrenamiento y la de validación ratifica la efectividad de la regularización por *Dropout* ($p = 0,2$) y la normalización por lotes (*Batch Normalization*), demostrando que el modelo no incurrió en sobreajuste (*overfitting*).

Por su parte, la reducción dinámica de la tasa de aprendizaje (α) es monitoreada en la Figura 17.

El programador ajustó progresivamente la tasa inicial de $\alpha = 10^{-3}$, reduciéndola por un factor de 0,5 cada vez que la pérdida de validación no mostró mejoras significativas durante 3 épocas consecutivas. Este mecanismo permitió explorar eficientemente la superficie de error en las etapas iniciales y realizar un ajuste fino en el espacio de parámetros conforme el optimizador se aproximaba al mínimo local óptimo.

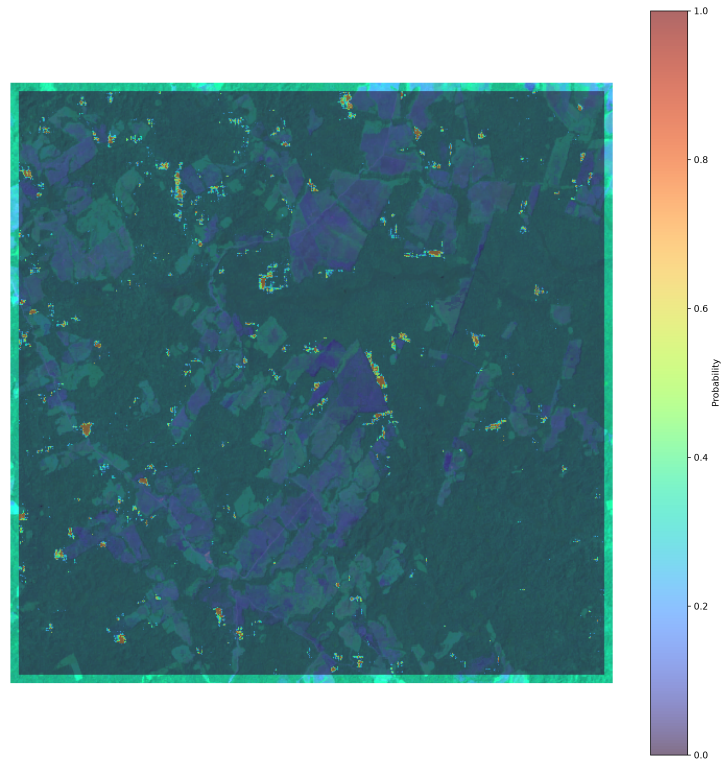


Figura 15: Superposición espacial de la deforestación proyectada para 2026 sobre el mapa de cobertura forestal histórica en Mapiripán (Meta).



Figura 16: Curvas de pérdida ($\mathcal{L}_{BCE}$) de entrenamiento y validación durante el proceso de optimización del modelo *Baseline*.

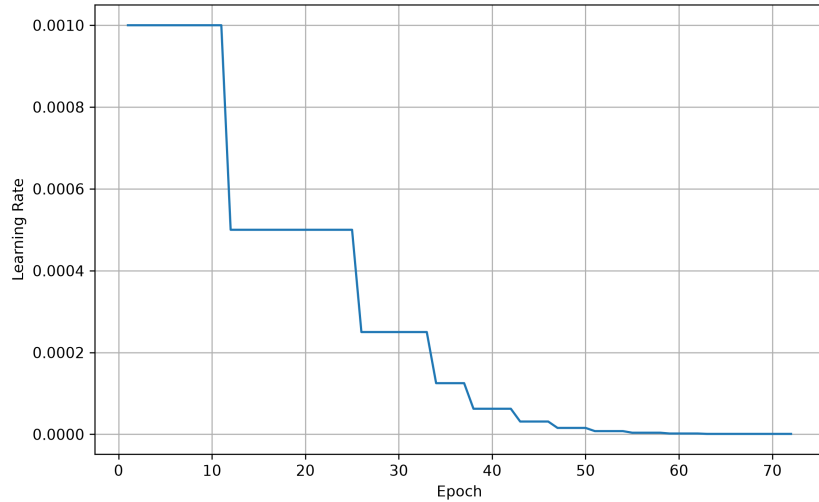


Figura 17: Evolución adaptativa de la tasa de aprendizaje (α) mediante el esquema de decaimiento por estancamiento en validación.

8.5. Monitoreo de Métricas de Validación

Para determinar de forma objetiva la detención temprana del entrenamiento y la selección de los mejores pesos (*checkpoint*), se monitoreó continuamente el desempeño del modelo sobre el conjunto de validación utilizando múltiples métricas cuantitativas: Precisión, Sensibilidad (*Recall*), F_1 -score y *ROC-AUC*. La Figura 18 muestra la trayectoria temporal de estos indicadores.

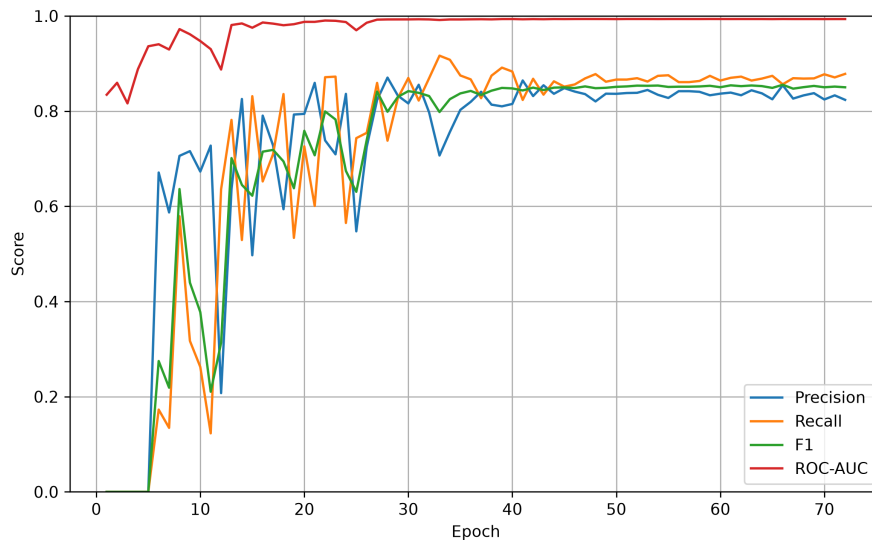


Figura 18: Evolución de las métricas de desempeño (*ROC-AUC*, F_1 -score, Precisión y Sensibilidad) en el conjunto de validación por época de entrenamiento.

El análisis de la Figura 18 revela que el valor de *ROC-AUC* exhibe un comportamiento altamente estable y ascendente desde las primeras épocas, superando rápidamente el valor de 0,98 y alcanzando su máximo absoluto en la época 47. En contraste, las métricas dependientes del umbral por defecto (tales como F_1 -score y Precisión) mostraron oscilaciones leves durante las primeras 30 épocas antes de estabilizarse. La coincidencia del pico máximo de *ROC-AUC* en la época 47 justifica formalmente la detención del entrenamiento mediante el criterio de paciencia

(25 épocas) y la selección de este punto de control para la inferencia definitiva.

8.6. Sensibilidad y Calibración del Umbral de Decisión

La salida de la función de activación sigmoide en la última capa de *MDFNet* proporciona un mapa continuo de probabilidades $p_i \in [0, 1]$ que representa la propensión a la deforestación de cada píxel. En escenarios con severo desbalance de clases o aplicaciones operativas, el umbral estándar $\tau = 0,50$ suele no ser óptimo. Por esta razón, se realizó un análisis de sensibilidad sobre el conjunto de validación variando τ de manera continua entre 0,0 y 1,0.

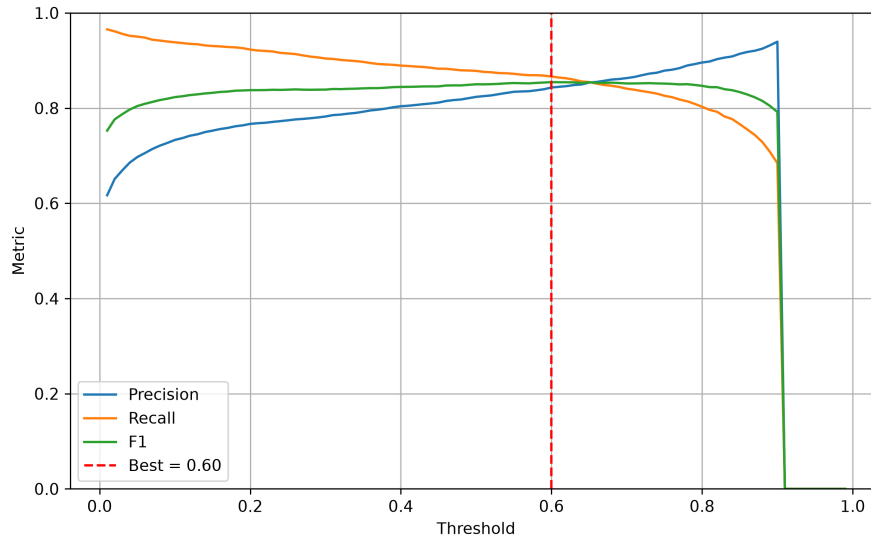


Figura 19: Curva de sensibilidad del umbral de probabilidad (τ) frente a las métricas de Precisión, Sensibilidad y F_1 -score en el conjunto de validación.

La Figura 19 ilustra el compromiso (*trade-off*) inherente entre la Precisión y la Sensibilidad:

- Umbrales conservadores bajos ($\tau < 0,40$) incrementan sustancialmente la Sensibilidad (capturando la mayoría de eventos de pérdida), a costa de reducir la Precisión por una mayor tasa de falsos positivos.
- Umbrales restrictivos altos ($\tau > 0,80$) garantizan una elevada Precisión, pero descartan eventos reales de deforestación al clasificarlos como falsos negativos.

El punto de intersección óptimo que maximiza el valor de F_1 -score se identificó cuantitativamente en $\tau^* = 0,60$. La elección de este umbral específico permitió equilibrar armónicamente ambas métricas en un nivel superior al 85 %, asegurando que la máscara binaria resultante no sobreestime el área deforestada ni omita frentes críticos de transformación territorial.

Referencias

- [1] Instituto de Hidrología, Meteorología y Estudios Ambientales (IDEAM), “Boletín de detección temprana de deforestación (dtd) no. 46,” 2026.
- [2] M. C. Hansen, P. V. Potapov, R. Moore, M. Hancher, S. A. Turubanova, A. Tyukavina, D. Thau, S. V. Stehman, S. J. Goetz, T. R. Loveland, *et al.*, “High-resolution global maps of 21st-century forest cover change,” *Science*, vol. 342, no. 6160, pp. 850–853, 2013.

- [3] J. G. C. Ball, K. Petrova, D. A. Coomes, and S. Flaxman, "Using deep convolutional neural networks to forecast spatial patterns of amazonian deforestation," *Methods in Ecology and Evolution*, vol. 13, no. 11, pp. 2622–2634, 2022.
- [4] Global Land Analysis and Discovery (GLAD), "Global forest change 2000–2020, version 1.8." <https://storage.googleapis.com/earthenginepartners-hansen/GFC-2020-v1.8/download.html>, 2021. Documentación oficial del conjunto de datos. Accedido: julio de 2026.
- [5] Global Land Analysis and Discovery (GLAD), "Global forest change viewer." <https://glad.earthengine.app/view/global-forest-change>, 2025. Visualización interactiva del conjunto de datos. Accedido: julio de 2026.
- [6] Global Land Analysis and Discovery (GLAD), "Global forest change 2000–2025, version 1.13." <https://storage.googleapis.com/earthenginepartners-hansen/GFC-2025-v1.13/download.html>, 2026. Última versión disponible del conjunto de datos. Accedido: julio de 2026.
- [7] T. Tadono, H. Ishida, F. Oda, S. Naito, K. Minakawa, and H. Iwamoto, "Precise global dem generation by alos prism," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. II-4, pp. 71–76, 2014.
- [8] Japan Aerospace Exploration Agency (JAXA), "Alos world 3d - 30m (aw3d30)." <https://www.eorc.jaxa.jp/ALOS/en/dataset/aw3d30/>, 2024. Página oficial del producto. Accedido: julio de 2026.
- [9] Japan Aerospace Exploration Agency (JAXA), "Alos world 3d (aw3d30) version 4.1." https://developers.google.com/earth-engine/datasets/catalog/JAXA_ALOS_AW3D30_V4_1, 2024. Documentación oficial del conjunto de datos disponible en Google Earth Engine. Accedido: julio de 2026.
- [10] J. G. C. Ball, K. Petrova, D. A. Coomes, and S. Flaxman, "Using deep convolutional neural networks to forecast spatial patterns of Amazonian deforestation: Supplementary materials," April 2022. Supplement to [3].
- [11] F.-F. Li, S. Savarese, and J. C. Niebles, "Cs231n: Convolutional neural networks for visual recognition," 2017. Course notes.
- [12] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015.
- [13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [14] X. Li, S. Chen, X. Hu, and J. Yang, "Understanding the disharmony between dropout and batch normalization by variance shift," *arXiv preprint arXiv:1801.05134*, 2018.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," in *Lecture Notes in Computer Science*, pp. 346–361, Springer, 2014.
- [16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [17] C. Olah, "Understanding lstm networks," Aug 2015.
- [18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.